

《数据库》

GaussDB 数据库开发指导手册



HUAWEI

华为技术有限公司



目录

前 言	3
简介	3
内容描述	3
实验环境说明	3
实验概览	4
1 GaussDB 数据库开发实验	5
1.1 创建和管理数据库	5
1.1.1 创建数据库	5
1.1.2 管理数据库	6
1.1.3 删除数据库	8
1.2 创建和管理用户	9
1.2.1 创建用户	9
1.2.2 管理用户	11
1.2.3 删除用户	20
1.3 创建和管理表	21
1.3.1 创建表	21
1.3.2 管理表	24
1.4 创建和管理其他数据库对象	36
1.4.1 创建和管理 schema	36
1.4.2 创建和管理分区表	44
1.4.3 创建和管理索引	50
1.4.4 创建和管理视图	59
1.4.5 创建和管理序列	63
1.4.6 创建和管理存储过程	66
1.4.7 创建和管理全局临时表	69
1.5 学校数据模型	73
1.5.1 关于本实验	73



1.5.2 关系模型 74

1.5.3 E-R 图..... 75

1.6 学校数据模型表操作 75

1.6.1 表的创建 75

1.6.2 表数据的插入 80

1.6.3 数据查询 87

前言

简介

本实验指导书主要目的是为了让读者了解 GaussDB 数据库的基本操作。

内容描述

本实验指导书通过用户管理、表管理、数据库对象等管理的操作，介绍在 DAS 环境下如何使用 GaussDB。

实验环境说明

- 组网说明

本实验环境为华为云 GaussDB 数据库。如果没有购买 GaussDB 则参考《01 GaussDB 数据库实验环境搭建指导手册》进行购买。

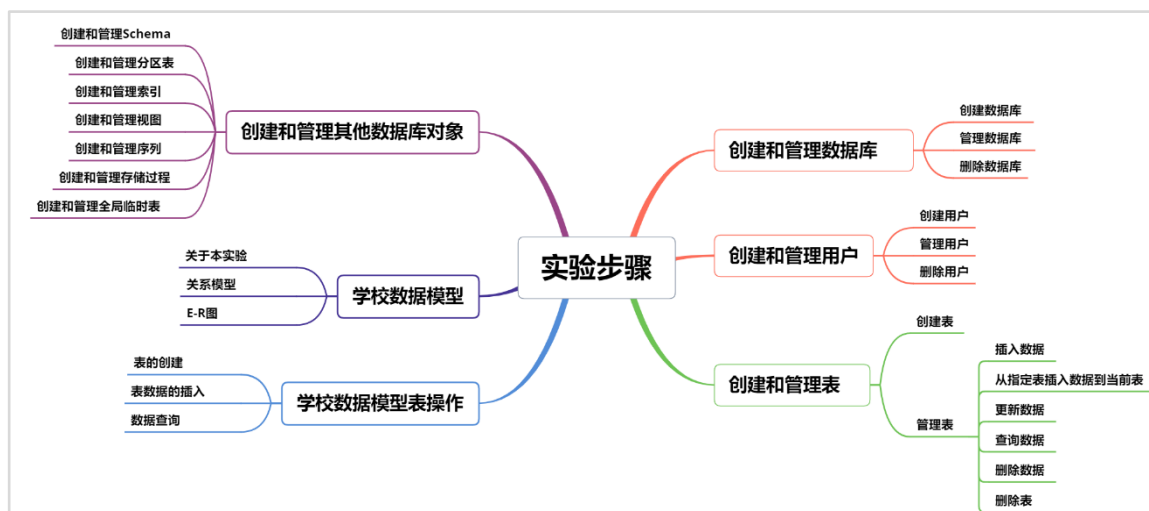
- 设备介绍

为了满足本实验需要，建议实验环境采用以下配置：

设备明细表

设备名称	设备型号	软件版本
数据库	GaussDB 4 核 16 GB	GaussDB 服务

实验概览



本实验概览图

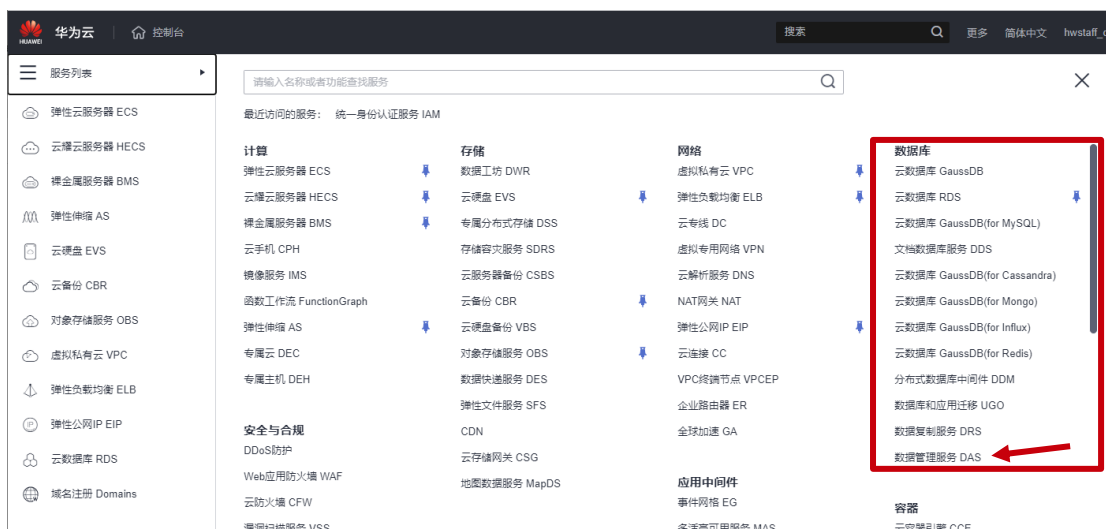
1 GaussDB 数据库开发实验

1.1 创建和管理数据库

1.1.1 创建数据库

步骤 1 进入数据库管理服务 DAS，登录 GaussDB。

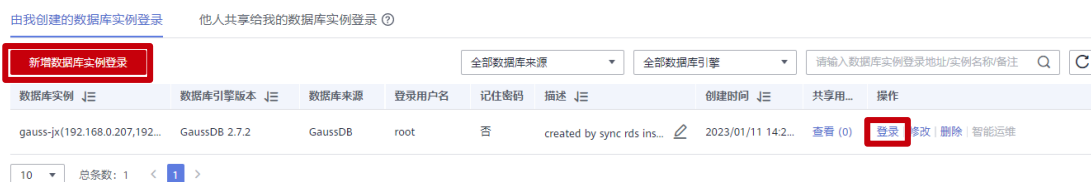
在服务列表，选择数据库中的数据库管理服务 DAS。



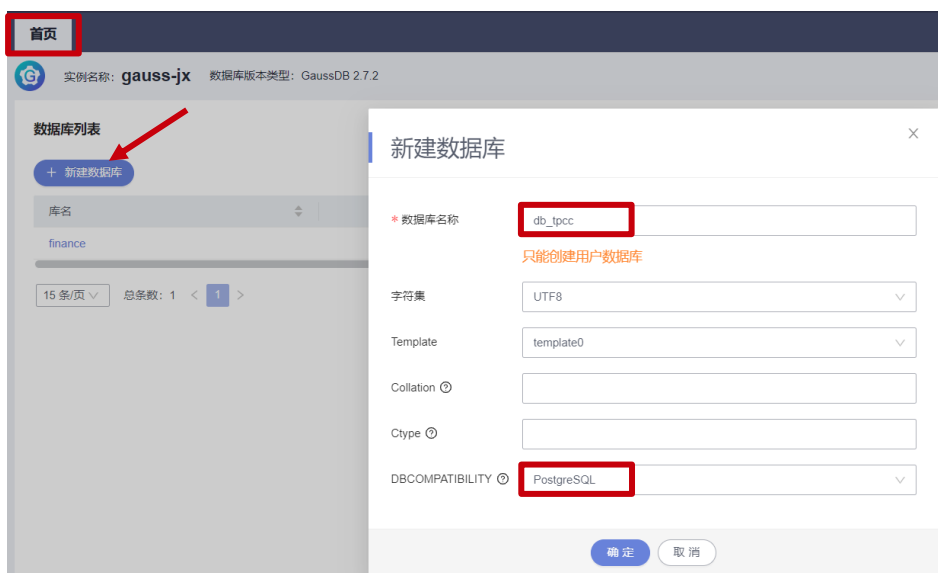
选择“进入开发工具”。



DAS 会自动为创建好的数据库实例建立一个连接，可以直接使用该连接登录。如果没有找到所需的连接，单击“新增数据库实例登录”。



步骤 2 创建一个新的数据库 db_tpcc。



注：

- Collation：排序规则，默认 en_US.utf8。请注意，不同的排序规则下，相同的比较其结果可能是不同的。例如，在 en_US.utf8 下，select 'a'>'A';为 false，但在'C'下，select 'a'>'A';为 true；如果从 oracle 迁移到 GaussDB，使用'C'才能得到一致的预期。支持的排序规则，可以查询系统表 pg_collation。
- Ctype：要在新数据库中使用的字符分类（LC_CTYPE）。这会影 响字符的类别，如小写、大写和数字。默认是使用模板数据库的字符分类。

创建成功：



1.1.2 管理数据库

步骤 1 单击库名或者操作->库管理，进入数据库管理页面，默认显示 Schema 列表。



可以切换查看用户 schema 或者系统 schema，也可以输入 schema 名进行搜索。



步骤 2 切换到对象列表进行查看，可以切换 schema 查看表、视图、存储过程等对象：



步骤 3 切换到元数据采集进行查看，单击“立即采集”，可以看到采集结果。

单击查看采集日志详情，查看日志详情：





1.1.3 删除数据库

用户可以使用 DROP DATABASE 命令删除数据库。此命令删除了数据库中的系统目录，并且删除了带有数据的磁盘上的数据库目录。用户必须是数据库的 owner 或者系统管理员才能删除数据库。当有人连接数据库时，删除操作会失败。删除数据库时请先连接到其他的数据库。

步骤 1 返回数据库实例首页，单击删除库：



步骤 2 确认删除数据库：



删除数据库成功：



1.2 创建和管理用户

1.2.1 创建用户

通过 CREATE USER 创建的用户，默认具有 LOGIN 权限；

通过 CREATE USER 创建用户的同时系统会在执行该命令的数据库中，为该用户创建一个同名的 SCHEMA；其他数据库中，则不自动创建同名的 SCHEMA；用户可使用 CREATE SCHEMA 命令，分别在其他数据库中，为该用户创建同名 SCHEMA。

系统管理员在普通用户同名 schema 下创建的对象，所有者为 schema 的同名用户（非系统管理员）。

步骤 1 选择 SQL 操作，单击 SQL 查询，进入 SQL 查询页面：



库名选择 postgres，Schema 选择 root：



步骤 2 创建用户。

创建用户 jim，登录密码为 Bigdata@123，在 SQL 查询页面，输入如下 SQL 语句：

```
CREATE USER jim PASSWORD 'Bigdata@123';
```

截图如下：



同样的下面语句也可以创建用户。

```
CREATE USER kim IDENTIFIED BY 'Bigdata@123';
```



如果创建有“创建数据库”权限的用户，则需要加 CREATEDB 关键字。

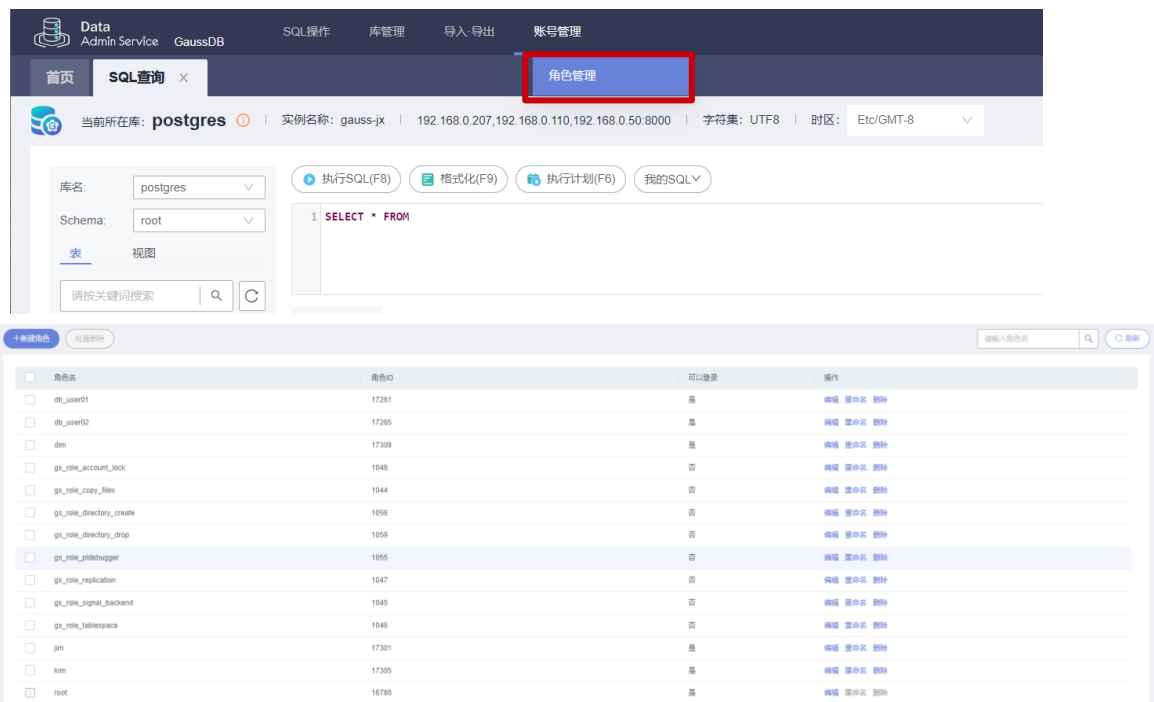
CREATE USER dim CREATEDB PASSWORD 'Bigdata@123';



The screenshot shows the GaussDB SQL interface. On the left, there are dropdowns for '库名' (Database Name) set to 'postgres' and 'Schema' set to 'root'. Below these are tabs for '表' (Table) and '视图' (View), and a search bar. The main area displays the SQL statement: `1 CREATE USER dim CREATEDB PASSWORD 'Bigdata@123';`. Below the statement, the 'SQL执行记录' (SQL Execution Record) tab is active, showing the execution details:
-----开始执行-----
【拆分SQL完成】：将执行SQL语句数量：（1条）
【执行SQL：（1）】
CREATE USER dim CREATEDB PASSWORD 'Bigdata@123';
执行成功，耗时：[40ms.]

1.2.2 管理用户

步骤 1 选择账号管理，单击角色管理，进入角色管理页面：



The screenshot shows the '角色管理' (Role Management) page in the GaussDB Admin Service. The page has a top navigation bar with 'Data Admin Service GaussDB' and tabs for 'SQL操作', '库管理', '导入/导出', and '账号管理'. The '账号管理' tab is selected. Below the navigation bar, there is a sidebar with '首页' and 'SQL查询' tabs, and a '角色管理' tab highlighted with a red box. The main area shows the '角色管理' page with a search bar and a table of roles.

角色名	角色ID	可以登录	操作
☐ db_user01	17261	是	编辑 重命名 删除
☐ db_user02	17265	是	编辑 重命名 删除
☐ dim	17309	是	编辑 重命名 删除
☐ gs_role_account_lock	1048	否	编辑 重命名 删除
☐ gs_role_copy_files	1044	否	编辑 重命名 删除
☐ gs_role_directory_create	1056	否	编辑 重命名 删除
☐ gs_role_directory_drop	1059	否	编辑 重命名 删除
☐ gs_role_pipelinebugger	1055	否	编辑 重命名 删除
☐ gs_role_replication	1047	否	编辑 重命名 删除
☐ gs_role_signal_backend	1045	否	编辑 重命名 删除
☐ gs_role_tablespace	1046	否	编辑 重命名 删除
☐ jim	17301	是	编辑 重命名 删除
☐ kim	17305	是	编辑 重命名 删除
☐ root	16789	是	编辑 重命名 删除

步骤 2 修改用户登录密码：

单击角色名 jim 操作列的“编辑”。

角色名	角色ID	可以登录	操作
☐ db_user01	17261	是	编辑 重命名 删除
☐ db_user02	17265	是	编辑 重命名 删除
☐ dim	17309	是	编辑 重命名 删除
☐ gs_role_account_lock	1946	否	编辑 重命名 删除
☐ gs_role_copy_files	1944	否	编辑 重命名 删除
☐ gs_role_directory_create	1956	否	编辑 重命名 删除
☐ gs_role_directory_drop	1959	否	编辑 重命名 删除
☐ gs_role_gatherbugger	1955	否	编辑 重命名 删除
☐ gs_role_replication	1947	否	编辑 重命名 删除
☐ gs_role_signal_backend	1945	否	编辑 重命名 删除
☐ gs_role_tablespace	1946	否	编辑 重命名 删除
☐ jim	17301	是	编辑 重命名 删除
☐ kim	17395	是	编辑 重命名 删除
☐ root	16789	是	编辑 重命名 删除

在密码框和确认密码框输入新密码，将用户 jim 的登录密码由 Bigdata@123 修改为 Abcd@123，单击保存：

返回角色管理列表 / 编辑角色 - jim

常规

所属角色组

权限

* 角色名

角色ID

可以登录

* 密码

* 确认密码

Node Group

连接限制数

生效时间

到期时间

可以创建数据库

可以创建角色

继承权限

jim

17301

☒

-1

请选择日期

请选择日期

☐

☐

☒

保存 取消

显示 SQL 预览，单击确定，修改成功。

SQL预览

1 ALTER ROLE "jim" PASSWORD '*****';

确定 取消

步骤 3 为用户 jim 追加可以创建数据库的权限。

返回角色管理列表 / 编辑角色 - jim

常规 所属角色组 权限

* 角色名

角色ID

可以登录

* 密码

* 确认密码

Node Group

连接限制数

生效时间

到期时间

可以创建数据库

可以创建角色

继承权限

jim

17301

☒

-1

请选择日期

请选择日期

☒

☐

☒

保存

取消

SQL预览

1 ALTER ROLE "jim" CREATEDB;

确定

取消

步骤 4 将 root 角色（用户）的权限赋予 jim 用户。选择“所属角色组”，勾选 root 后面的“授予”。

返回角色管理列表 / 编辑角色 - jim

常规 所属角色组 权限

角色名

授予

具有授予权限

db_user01

☐

☐

db_user02

☐

☐

dim

☐

☐

gs_role_account_lock

☐

☐

gs_role_copy_files

☐

☐

gs_role_directory_create

☐

☐

gs_role_directory_drop

☐

☐

gs_role_pidbugger

☐

☐

gs_role_replication

☐

☐

gs_role_signal_backend

☐

☐

gs_role_tablespace

☐

☐

jim

☐

☐

kim

☐

☐

root

☒

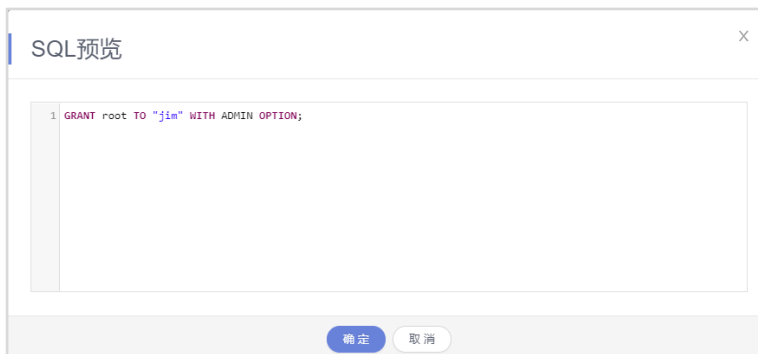
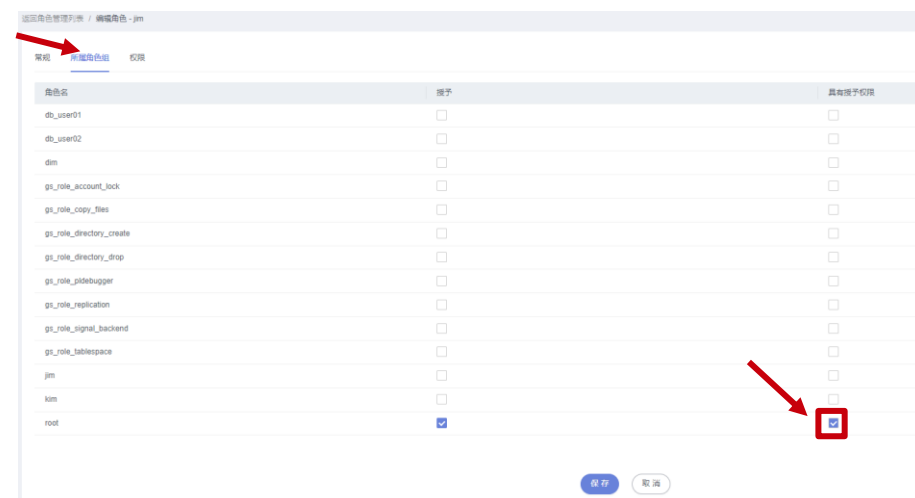
☐

保存

取消



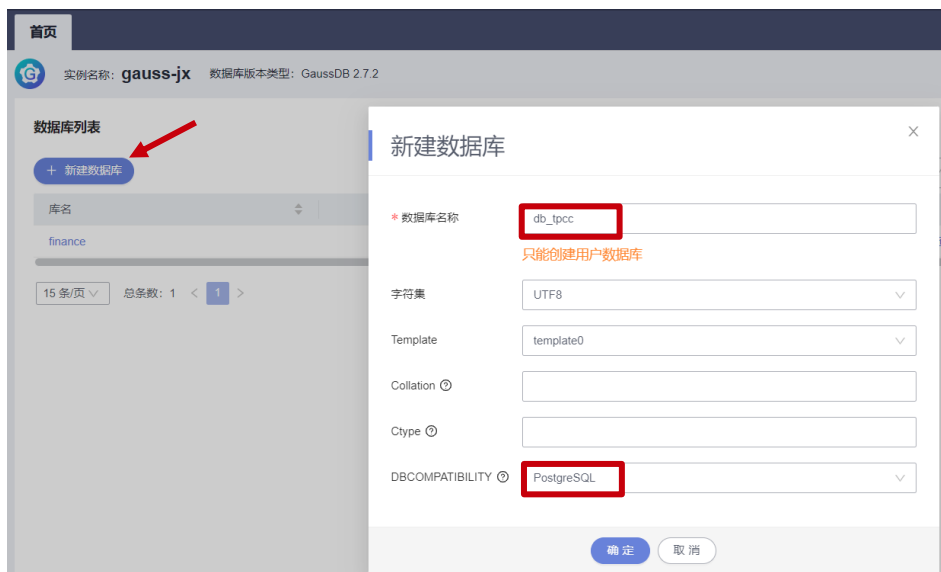
连带授权：



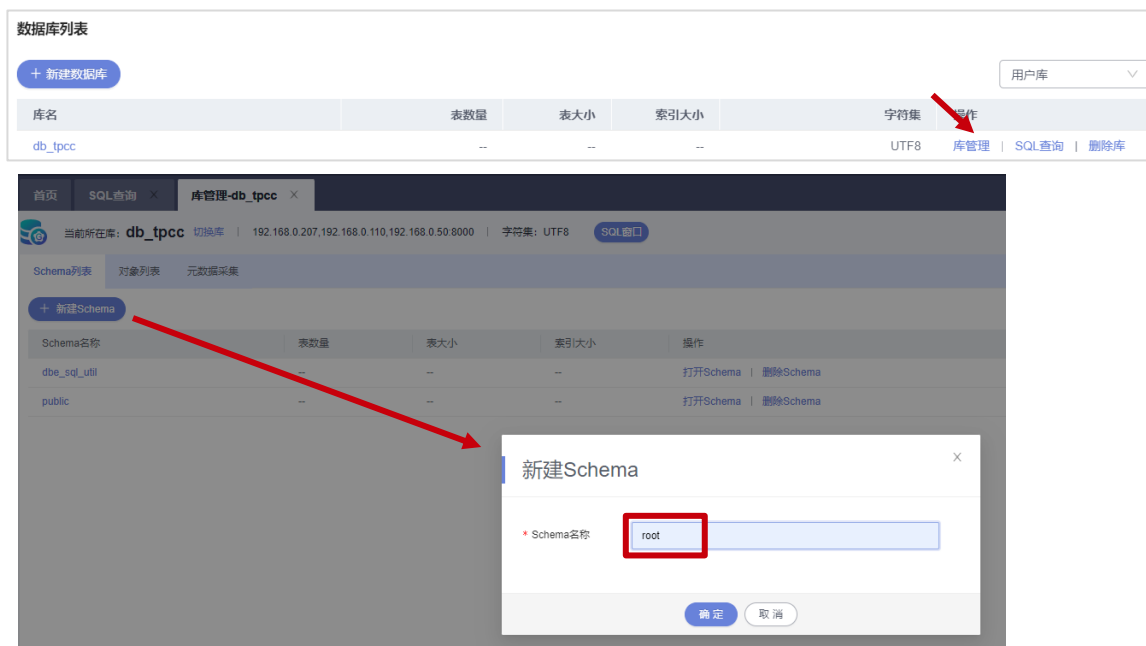
授权后 jim 用户可以通过 SQL 语句将 root 角色授予其他用户或角色。例如在 SQL 窗口中执行：GRANT root TO kim; 将 root 角色授予 kim 用户。

步骤 5 设置用户权限。

创建数据库 db_tpcc。



单击库管理，创建 root 用户的同名 Schema



创建成功后，单击 SQL 窗口。



Schema 选为 root，用以下语句在窗口中创建一张 TEST 表，具体如下：

```
CREATE TABLE test(testid int);
```



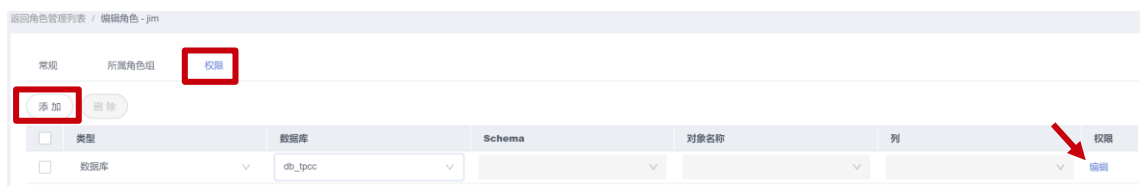
单击“账号管理->角色管理”；



单击角色名 jim 操作列的“编辑”。

☐	角色名	角色ID	可以登录	操作
☐	db_user01	17261	是	编辑 重命名 删除
☐	db_user02	17265	是	编辑 重命名 删除
☐	dim	17309	是	编辑 重命名 删除
☐	gs_role_account_lock	1048	否	编辑 重命名 删除
☐	gs_role_copy_files	1044	否	编辑 重命名 删除
☐	gs_role_directory_create	1056	否	编辑 重命名 删除
☐	gs_role_directory_drop	1059	否	编辑 重命名 删除
☐	gs_role_plidebugger	1055	否	编辑 重命名 删除
☐	gs_role_replication	1047	否	编辑 重命名 删除
☐	gs_role_signal_backend	1045	否	编辑 重命名 删除
☐	gs_role_tablespace	1046	否	编辑 重命名 删除
☐	jim	17301	是	编辑 重命名 删除
☐	kim	17305	是	编辑 重命名 删除
☐	root	16780	是	编辑 重命名 删除

单击“权限->添加”，类型选择数据库，数据库选择 db_tpcc，然后单击“编辑”。



勾选授予 CONNECT 权限：

请选择权限

权限	授予 ☐	具有授予权限 ☐
CONNECT	☒	☐
CREATE	☐	☐
TEMPORARY	☐	☐

确定
取消

常规
所属角色组
权限

添加
删除

类型	数据库	Schema	对象名称	列	权限
数据库	db_tpcc				CONNECT 编辑

再次单击添加，类型选择 Schema，数据库选择 db_tpcc，Schema 选择 root，单击编辑：

常规
所属角色组
权限

添加
删除

类型	数据库	Schema	对象名称	列	权限
Schema	db_tpcc	root			编辑
数据库	db_tpcc				CONNECT 编辑

勾选授予 USAGE 权限，单击确定：

请选择权限

权限	授予 ☐	具有授予权限 ☐
CREATE	☐	☐
USAGE	☒	☐

确定
取消

再次单击添加，类型选择表，数据库选择 db_tpcc，Schema 选择 root，对象名称选择 test，单击编辑

添加
删除

类型	数据库	Schema	对象名称	列	权限
表	db_tpcc	root	test		编辑

勾选授予 SELECT 权限：

请选择权限

权限	授予 ☐	具有授予权限 ☐
DELETE	☐	☐
INSERT	☐	☐
REFERENCES	☐	☐
SELECT	☒	☐
TRIGGER	☐	☐
TRUNCATE	☐	☐
UPDATE	☐	☐

确定
取消

添加完成后选择保存，单击确定后，权限添加完毕。

SQL预览

```

1 /* Switch to database db_tpcc */
2 GRANT CONNECT ON DATABASE "db_tpcc" TO "jim";
3 GRANT USAGE ON SCHEMA "root" TO "jim";
4 GRANT SELECT ON TABLE "root"."test" TO "jim";

```

确定
取消

Data Admin Service GaussDB SQL操作 库管理 导入/导出 账号管理

角色管理

创建用户 jim 成功

角色名	角色ID	可以登录	操作
☐ db_user01	17261	是	编辑 重命名 删除
☐ db_user02	17265	是	编辑 重命名 删除
☐ dm	17309	是	编辑 重命名 删除
☐ ga_role_account_lock	1048	否	编辑 重命名 删除
☐ ga_role_copy_files	1044	否	编辑 重命名 删除
☐ ga_role_directory_create	1056	否	编辑 重命名 删除
☐ ga_role_directory_drop	1059	否	编辑 重命名 删除
☐ ga_role_pidebugger	1055	否	编辑 重命名 删除
☐ ga_role_replication	1047	否	编辑 重命名 删除
☐ ga_role_signal_backend	1045	否	编辑 重命名 删除
☐ ga_role_tablespace	1046	否	编辑 重命名 删除
☐ jim	17301	是	编辑 重命名 删除
☐ km	17305	是	编辑 重命名 删除
☐ root	16780	是	编辑 重命名 删除

步骤 6 验证用户权限

单击右上角账户名，选择切换连接



数据库登录

当前实例: gauss-jx

gauss-jx - 192.168.0.207,192.168.0.110,192.168.0.50:8000

当前最多展示100条数据库登录，可前往[数据库登录列表](#)查看更多。

* 数据库名称:

postgres

* 登录用户名:

jim

密码:

☐ 记住密码

☐ SQL执行记录 ②

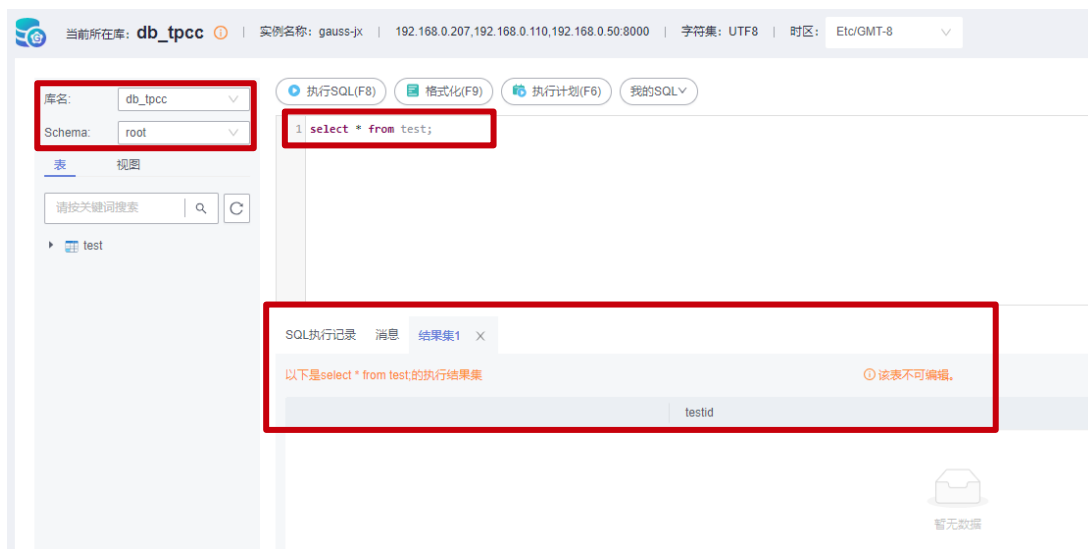
开启此项后，您可以在DAS中，方便的查看到您的SQL窗口执行历史记录，并且可以直接再次执行，无需重复输入。

登录

选择 db_tpcc 数据库的 SQL 查询，确认 schema 选择为 root，输入查询语句：

```
select * from test;
```

可以查询，权限赋予成功，查询结果如下：



1.2.3 删除用户

步骤 1 切换连接到 root 用户。

数据库登录

当前实例: gauss-jx

gauss-jx - 192.168.0.207,192.168.0.110,192.168.0.50:8000

当前最多展示100条数据库登录, 可前往[数据库登录列表](#)查看更多。

* 数据库名称:

postgres

* 登录用户名:

root

密码:

☒ 记住密码

同意用户名及密码记录到DAS系统中, 如不再需要, 可以在[数据库登录列表](#)页面中删除。

☒ SQL执行记录 ②

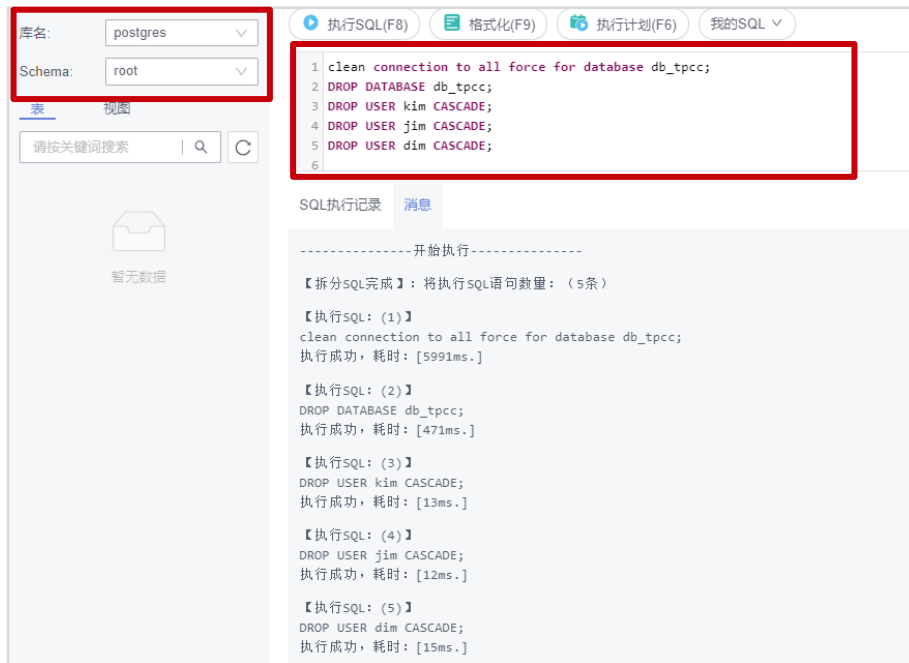
开启此项后, 您可以在DAS中, 方便的查看到您的SQL窗口执行历史记录, 并且可以直接再次执行, 无需重复输入。

登录

步骤 2 进入 SQL 查询页面输入以下代码，进行级联删除。注意：左侧数据库切换为 postgres，Schema 选择 root。

```
clean connection to all force for database db_tpcc;
DROP DATABASE db_tpcc;
DROP USER kim CASCADE;
DROP USER jim CASCADE;
DROP USER dim CASCADE;
```

删除成功。

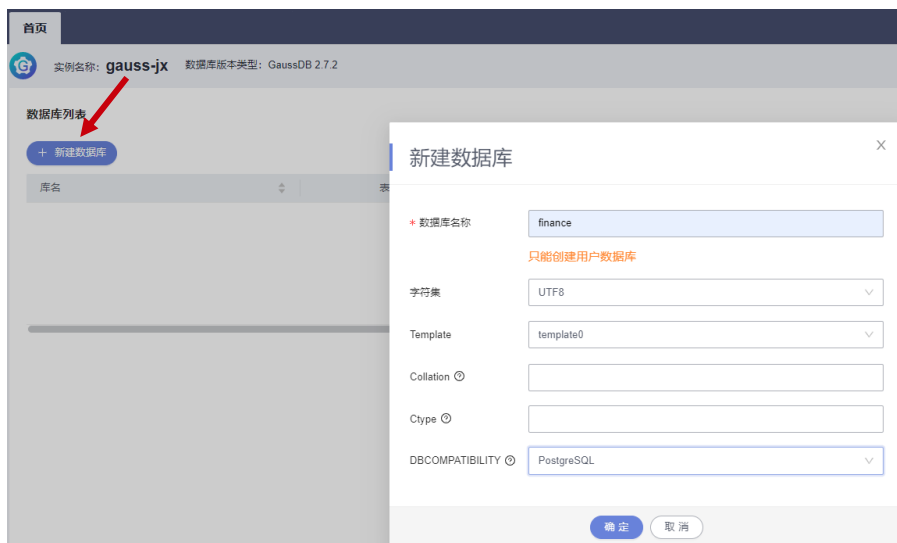


1.3 创建和管理表

1.3.1 创建表

表是建立在数据库中的，在不同的数据库中可以存放相同的表。甚至可以通过使用模式在同一个数据库中创建相同名称的表。

步骤 1 在首页中，创建数据库 finance。



在数据库列表页面，单击刚创建的数据库名 finance 进入库管理页面，然后在 finance 数据库中创建与 root 账户同名的 schema。



步骤 2 单击 SQL 窗口，进入 SQL 查询界面：



单击“SQL 窗口”后，具体如下：



步骤 3 在 SQL 查询页面中 SQL 区，执行如下命令创建表。

```
CREATE TABLE customer_t1
(
  c_customer_sk      integer,
  c_customer_id      char(5),
  c_first_name       char(6),
  c_last_name        char(8)
);
```

执行截图如下：



其中 c_customer_sk 、 c_customer_id、 c_first_name 和 c_last_name 是表的字段名，integer、char(5)、char(6)和 char(8)分别是这四个字段的数据类型。

1.3.2 管理表

1.3.2.1 插入数据

步骤 1 向表 customer_t1 中插入一行数据。

- 数据值是按照这些字段在表中出现的顺序列出的，并且用逗号分隔。通常数据值是文本（常量），但也允许使用标量表达式。

```
INSERT INTO customer_t1(c_customer_sk, c_customer_id, c_first_name) VALUES (3769, 'hello', 'Grace');
```

执行截图如下：



- 如果用户已经知道表中字段的顺序，也可无需列出表中的字段。例如以下命令与上面的命令效果相同，同样可以插入一行数据。

```
INSERT INTO customer_t1 VALUES (3770, 'hi', 'Lucy');
```

执行截图如下：



- 如果用户不知道所有字段的数值，可以忽略其中的一些。没有数值的字段将被填充为字段的缺省值。例如：

```
INSERT INTO customer_t1 (c_customer_sk, c_first_name) VALUES (3771, 'Tom');
```

执行截图如下：



或

```
INSERT INTO customer_t1 VALUES (3772, 'hello');
```

执行截图如下：

执行SQL(F8)
格式化(F9)
执行计划(F6)
我的SQL ▾

```
1 INSERT INTO customer_t1 VALUES (3772, 'hello');
```

SQL执行记录
消息

```

-----开始执行-----

【拆分SQL完成】：将执行SQL语句数量：（1条）

【执行SQL：（1）】
INSERT INTO customer_t1 VALUES (3772, 'hello');
执行成功，耗时：[9ms.]

```

- 用户也可以对独立的字段或者整个行明确缺省值：

```
INSERT INTO customer_t1 (c_customer_sk, c_customer_id, c_first_name) VALUES (3773, 'hello', DEFAULT);
```

执行截图如下：

执行SQL(F8)
格式化(F9)
执行计划(F6)
我的SQL ▾

```
1 INSERT INTO customer_t1 (c_customer_sk, c_customer_id, c_first_name) VALUES (3773, 'hello', DEFAULT);
```

SQL执行记录
消息

```

-----开始执行-----

【拆分SQL完成】：将执行SQL语句数量：（1条）

【执行SQL：（1）】
INSERT INTO customer_t1 (c_customer_sk, c_customer_id, c_first_name) VALUES (3773, 'hello', DEFAULT);
执行成功，耗时：[9ms.]

```

或

```
INSERT INTO customer_t1 DEFAULT VALUES;
```

执行截图如下：

执行SQL(F8)
格式化(F9)
执行计划(F6)
我的SQL ▾

```
1 INSERT INTO customer_t1 DEFAULT VALUES;
```

SQL执行记录
消息

```

-----开始执行-----

【拆分SQL完成】：将执行SQL语句数量：（1条）

【执行SQL：（1）】
INSERT INTO customer_t1 DEFAULT VALUES;
执行成功，耗时：[8ms.]

```

步骤 2 向表中插入多行数据

命令如下：

```
INSERT INTO customer_t1 (c_customer_sk, c_customer_id, c_first_name) VALUES
(6885, 'maps', 'Joes'),
(4321, 'tpcds', 'Lily'),
(9527, 'world', 'James');
```

执行截图如下：

执行SQL(F8)
格式化(F9)
执行计划(F6)
我的SQL ▾

```

1 INSERT INTO customer_t1 (c_customer_sk, c_customer_id, c_first_name) VALUES
2   (6885, 'maps', 'Joes'),
3   (4321, 'tpcds', 'Lily'),
4   (9527, 'world', 'James');
```

SQL执行记录
消息

-----开始执行-----

【拆分SQL完成】：将执行SQL语句数量：（1条）

【执行SQL：（1）】

```

INSERT INTO customer_t1 (c_customer_sk, c_customer_id, c_first_name) VALUES
  (6885, 'maps', 'Joes'),
  (4321, 'tpcds', 'Lily'),
  (9527, 'world', 'James');
```

执行成功，耗时：[9ms.]

如果需要向表中插入多条数据，除此命令外，也可以多次执行插入一行数据命令实现。但是建议使用此命令可以提升效率。

步骤 3 查询表中的数据：

```
SELECT * FROM customer_t1;
```

执行截图如下：

执行SQL(F8)
格式化(F9)
执行计划(F6)
我的SQL ▾

```

1 SELECT * FROM customer_t1;
```

SQL执行记录
消息
结果集1 ×

以下是SELECT * FROM customer_t1:的执行结果集

该表不可编辑。

	c_customer_sk	c_customer_id	c_first_name
1	3769	hello	Grace
2	3770	hi	Lucy
3	3771		Tom
4	3772	hello	
5	3773	hello	

1.3.2.2 从指定表插入数据到当前表

步骤 1 如果从指定表插入数据到当前表，例如在数据库中创建了一个表 customer_t1 的备份表 customer_t2，现在需要将表 customer_t1 中的数据插入到表 customer_t2 中，则可以执行如下命令。

```
CREATE TABLE customer_t2
(
    c_customer_sk      integer,
    c_customer_id      char(5),
    c_first_name       char(6),
    c_last_name        char(8)
);
```

执行截图如下：



步骤 2 插入数据：

```
INSERT INTO customer_t2 SELECT * FROM customer_t1;
```

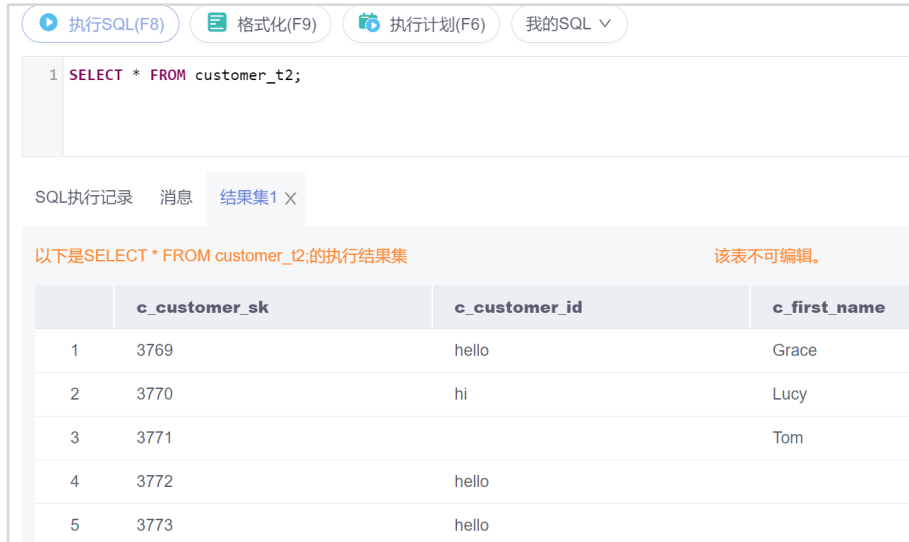
执行截图如下：



步骤 3 查询表中的数据：

```
SELECT * FROM customer_t2;
```

执行截图如下：



SQL执行记录 消息 结果集1 X

以下是SELECT * FROM customer_t2;的执行结果集 该表不可编辑。

	c_customer_sk	c_customer_id	c_first_name
1	3769	hello	Grace
2	3770	hi	Lucy
3	3771		Tom
4	3772	hello	
5	3773	hello	

和 customer_t1 表中数据一致。

步骤 4 删除备份表：

```
DROP TABLE customer_t2 CASCADE;
```

执行截图如下：



SQL执行记录 消息

-----开始执行-----

【拆分SQL完成】：将执行SQL语句数量：（1条）

【执行SQL：（1）】

DROP TABLE customer_t2 CASCADE;

执行成功，耗时：[43ms.]

1.3.2.3 更新数据

修改已经存储在数据库中数据的行为叫做更新。用户可以更新单独一行，所有行或者指定的部分行。还可以独立更新每个字段，而其他字段则不受影响。

- 将表 customer_t1 中 c_customer_sk 为 9527 的数据的 c_first_name 字段重新定义为 John:

```
UPDATE customer_t1 SET c_first_name = 'John' WHERE c_customer_sk = 9527;
```

执行截图如下：



- 目前版本中分布键列不允许被更新，例如将表 customer_t1 中 c_customer_sk 为 9527 的字段重新定义为 9876，就会报错：

```
UPDATE customer_t1 SET c_customer_sk = 9876 WHERE c_customer_sk = 9527;
```

执行截图如下：

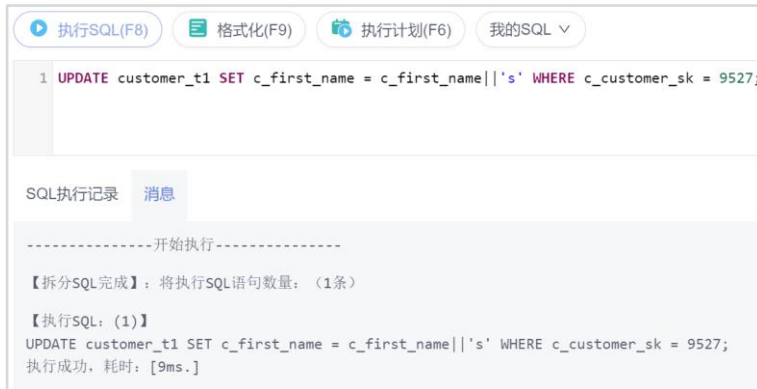


这里的表名称也可以使用模式名修饰，否则会从默认的模式路径找到这个表。SET 后面紧跟字段和新的字段值。新的字段值不仅可以是常量，也可以是变量表达式。

- 比如，将表 customer_t1 中 c_customer_sk 为 9527 的数据的 c_first_name 字段内容后加上's'：

```
UPDATE customer_t1 SET c_first_name = c_first_name||'s' WHERE c_customer_sk = 9527;
```

执行截图如下：



- 用户可以在一个 UPDATE 命令中更新更多的字段，方法是在 SET 子句中列出更多赋值，比如：

```
UPDATE customer_t1 SET c_customer_id = 'Admin', c_first_name = 'Local' WHERE c_customer_sk = 4321;
```

执行截图如下：



1.3.2.4 查询数据

- 查询所有数据，在“SQL 查询”页面输入如下语句，查询表 customer_t1 的数据量。

```
SELECT count(*) FROM customer_t1;
```

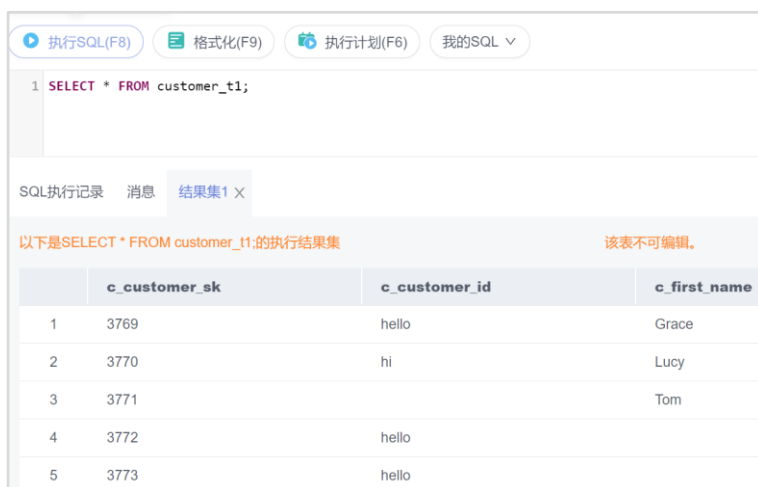
截图如下：



- 在“SQL 查询”页面输入如下语句，查询表 customer_t1 的所有数据。

```
SELECT * FROM customer_t1;
```

截图如下：



SQL执行记录 消息 结果集1 X

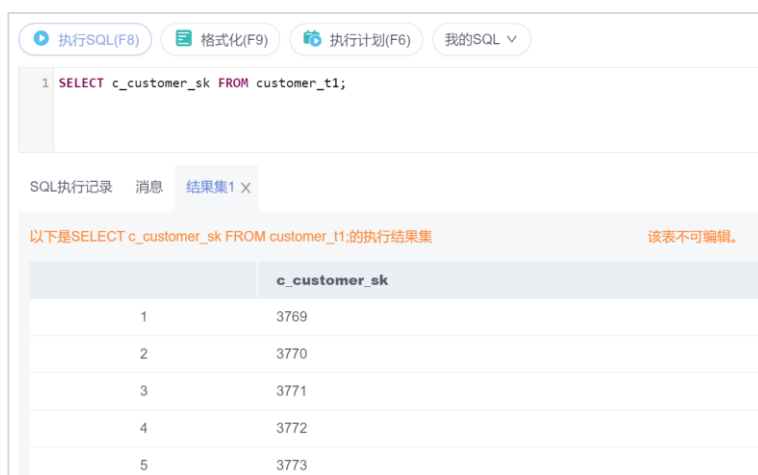
以下是SELECT * FROM customer_t1的执行结果集 该表不可编辑。

	c_customer_sk	c_customer_id	c_first_name
1	3769	hello	Grace
2	3770	hi	Lucy
3	3771		Tom
4	3772	hello	
5	3773	hello	

- 在“SQL 查询”页面输入如下语句，只查询字段 c_customer_sk 的数据。

```
SELECT c_customer_sk FROM customer_t1;
```

截图如下：



SQL执行记录 消息 结果集1 X

以下是SELECT c_customer_sk FROM customer_t1的执行结果集 该表不可编辑。

	c_customer_sk
1	3769
2	3770
3	3771
4	3772
5	3773

- 在“SQL 查询”页面输入如下语句，只过滤字段 c_customer_sk 的重复数据。

```
SELECT DISTINCT( c_customer_sk ) FROM customer_t1;
```

截图如下：

执行SQL(F8)
格式化(F9)
执行计划(F6)
我的SQL v

```
1 SELECT DISTINCT( c_customer_sk ) FROM customer_t1;
```

SQL执行记录 消息 结果集1 x

以下是SELECT DISTINCT(c_customer_sk) FROM customer_t1;的执行结果集

	c_customer_sk
1	3773
2	3770
3	3769

在“SQL 查询”页面输入如下语句，查询字段 c_customer_sk 为 3769 的所有数据。

```
SELECT * FROM customer_t1 WHERE c_customer_sk = 3769;
```

截图如下：

执行SQL(F8)
格式化(F9)
执行计划(F6)
我的SQL v

```
1 SELECT * FROM customer_t1 WHERE c_customer_sk = 3769;
```

SQL执行记录 消息 结果集1 x

以下是SELECT * FROM customer_t1 WHERE c_customer_sk = 3769;的执行结果集

该表不可编辑。

	c_customer_sk	c_customer_id	c_first_name
1	3769	hello	Grace

- 在“SQL 查询”页面输入如下语句，按照字段 c_customer_sk 进行排序。

```
SELECT * FROM customer_t1 ORDER BY c_customer_sk;
```

截图如下：

执行SQL(F8)
格式化(F9)
执行计划(F6)
我的SQL v

```
1 SELECT * FROM customer_t1 ORDER BY c_customer_sk;
```

SQL执行记录 消息 结果集1 x

以下是SELECT * FROM customer_t1 ORDER BY c_customer_sk;的执行结果集

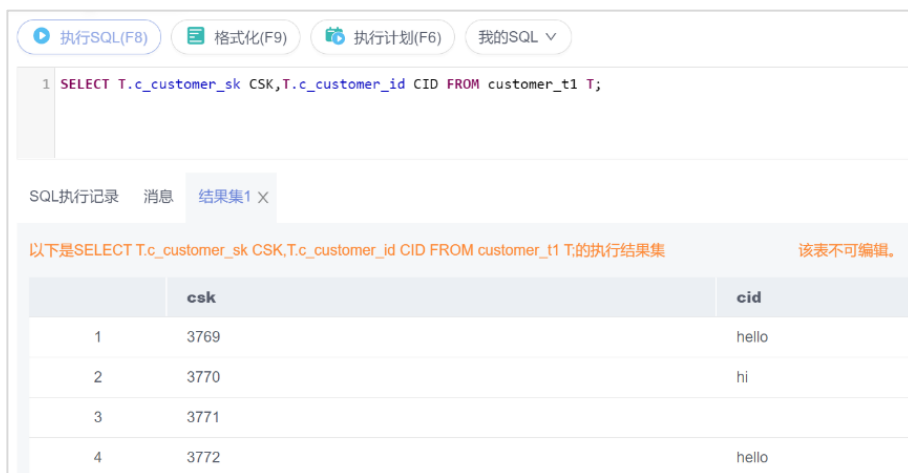
该表不可编辑。

	c_customer_sk	c_customer_id	c_first_name
1	3769	hello	Grace
2	3770	hi	Lucy
3	3771		Tom
4	3772	hello	

- 在“SQL 查询”页面输入如下语句，使用别名进行查询(CNB、CSK、CID 为列别名，T 为表别名)。

```
SELECT T.c_customer_sk CSK,T.c_customer_id CID FROM customer_t1 T;
```

截图如下：



SQL执行记录 消息 结果集1 X

以下是SELECT T.c_customer_sk CSK,T.c_customer_id CID FROM customer_t1 T;的执行结果集 该表不可编辑。

	csk	cid
1	3769	hello
2	3770	hi
3	3771	
4	3772	hello

1.3.2.5 删除数据

在使用表的过程中，可能会需要删除已过期的数据，删除数据必须从表中整行的删除。

- 在“SQL 查询”页面输入如下语句，使用 DELETE 命令删除行，如果删除表 customer_t1 中所有 c_customer_sk 为 3769 的记录：

```
DELETE FROM customer_t1 WHERE c_customer_sk = 3769;
```

截图如下：



SQL执行记录 消息

-----开始执行-----

【拆分SQL完成】：将执行SQL语句数量：（1条）

【执行SQL：（1）】

DELETE FROM customer_t1 WHERE c_customer_sk = 3769;

执行成功，耗时：[10ms.]

- 如果执行如下命令之一，会删除表中所有的行。

```
DELETE FROM customer_t1;
```

或：

```
TRUNCATE TABLE customer_t1;
```

截图如下：

执行SQL(F8)
格式化(F9)
执行计划(F6)
我的SQL v

```
1 DELETE FROM customer_t1;
```

SQL执行记录
消息

-----开始执行-----

【拆分SQL完成】：将执行SQL语句数量：（1条）

【执行SQL：（1）】
DELETE FROM customer_t1;
执行成功，耗时：[9ms.]

执行SQL(F8)
格式化(F9)
执行计划(F6)
我的SQL v

SQL执行记录
消息

1.3.2.6 删除表

步骤 1 切换到库管理页面->对象列表，查看表列表：

首页
库管理-finance x
SQL查询 x

当前所在库: finance 切换库
192.168.0.207,192.168.0.110,192.168.0.50:8000
字符集: UTF8
SQL窗口

Schema列表
对象列表
元数据采集

对象列表数据来自DAS后台定时采集任务，最近一次采集时间: 2023-01-19 04:00:36 立即采集 清空采集数据

表
视图
存储过程

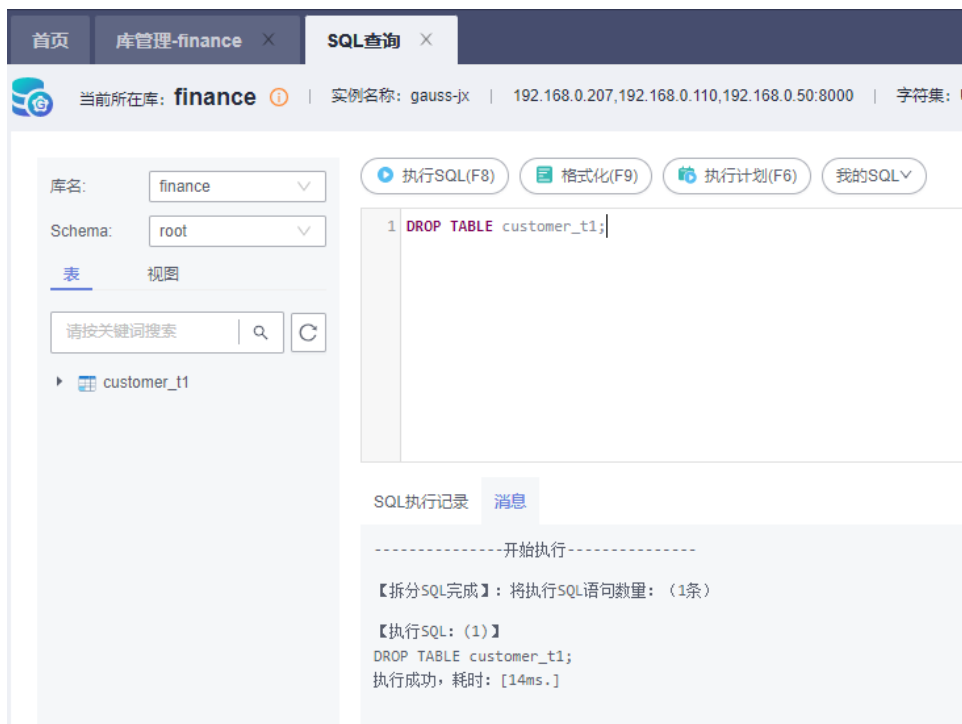
Schema: root

表名	行数	表大小	索引大小	字符集	操作
customer_t1	0	0B	0B	UTF8	SQL查询 打开表 查看表详情

步骤 2 切换到 SQL 查询页面，在“SQL 查询”页面输入如下语句，删除表：

DROP TABLE customer_t1;

截图如下：



步骤 3 切换到库管理页面->对象列表，刷新页面，查看表列表，表删除成功：



1.4 创建和管理其他数据库对象

1.4.1 创建和管理 schema

- Schema 又称作模式。通过管理 Schema，允许多个用户使用同一数据库而不相互干扰，可以将数据库对象组织成易于管理的逻辑组，同时便于将第三方应用添加到相应的 Schema 下而不引起冲突。
- 每个数据库包含一个或多个 Schema。数据库中的每个 Schema 包含表和其他类型的对象。数据库创建初始，默认具有一个名为 public 的 Schema，且所有用户都拥有此 Schema

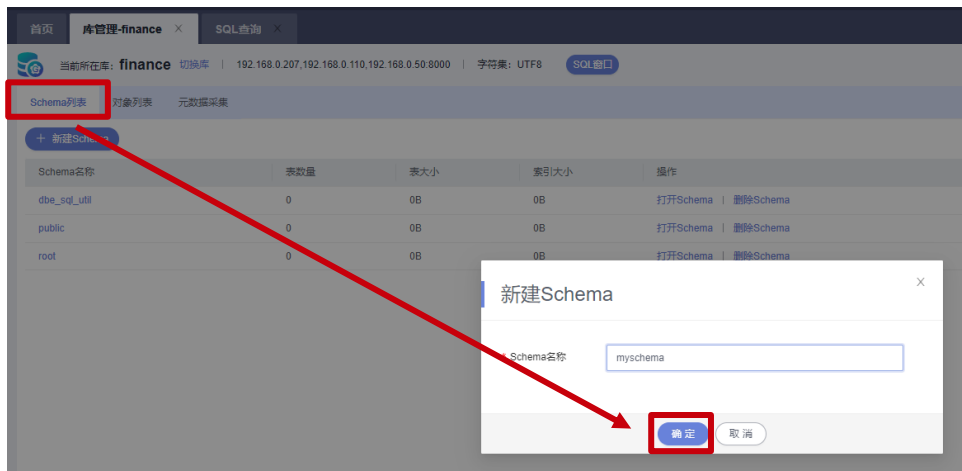
的权限。可以通过 Schema 分组数据库对象。Schema 类似于操作系统目录，但 Schema 不能嵌套。

- 相同的数据库对象名称可以应用在同一数据库的不同 Schema 中，而没有冲突。例如，a_schema 和 b_schema 都可以包含名为 mytable 的表。具有所需权限的用户可以访问数据库的多个 Schema 中的对象。

1.4.1.1 创建 schema

进入 finance 数据库的库管理界面，创建一个 Schema，并在 Schema 创建一个表。

步骤 1 创建 myschema 模式。



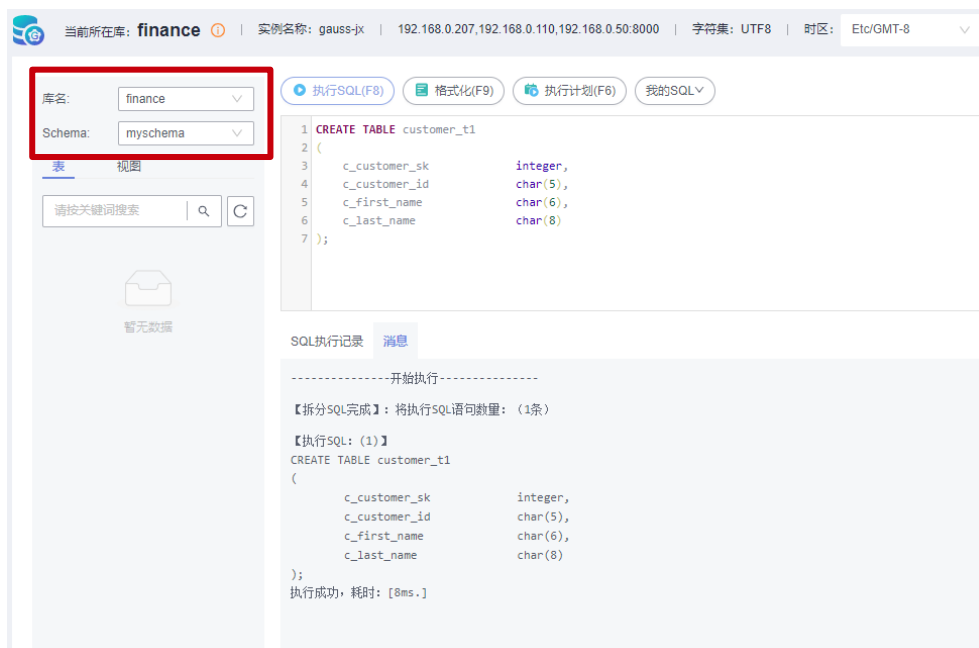
Schema 列表信息显示如下，成功创建一个名为 myschema 的 Schema。

Schema名称	表数量	表大小	索引大小	操作
db_e_sql_util	0	0B	0B	打开Schema 删除Schema
public	0	0B	0B	打开Schema 删除Schema
root	0	0B	0B	打开Schema 删除Schema
myschema	0	0B	0B	打开Schema 删除Schema

步骤 2 在 myschema Schema 下创建 customer_t1 表：

```
CREATE TABLE customer_t1
(
  c_customer_sk      integer,
  c_customer_id      char(5),
  c_first_name       char(6),
  c_last_name        char(8)
);
```

截图显示如下：



1.4.1.2 管理 Schema

1.4.1.2.1 schema 的权限控制

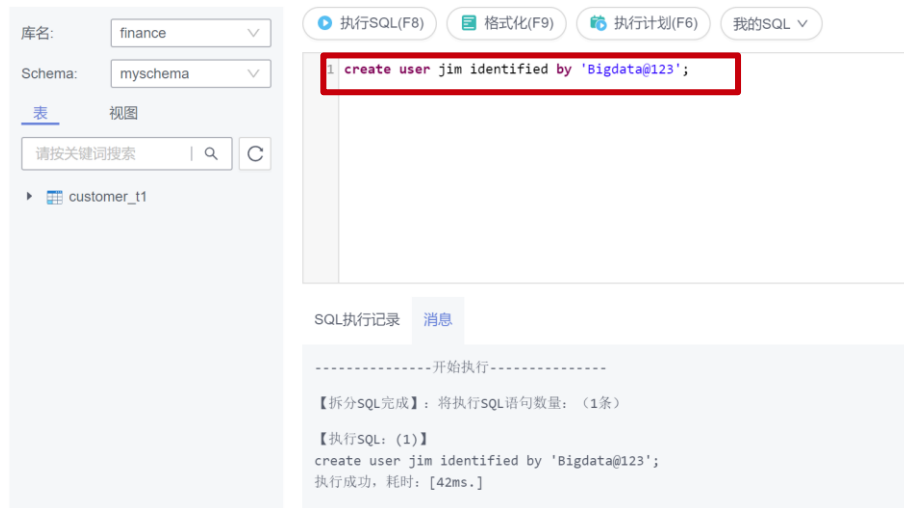
- 默认情况下，用户只能访问属于自己的 schema 中的数据库对象。如果需要访问其他 schema 的对象，则该 schema 的所有者应该赋予他对该 schema 的 USAGE 权限。通过将模式的 CREATE 权限授予某用户，被授权用户就可以在此模式中创建对象。
- 注意默认情况下，所有角色都拥有在 public 模式上的 USAGE 权限，但是普通用户没有 public 模式上的 CREATE 权限。

下面的操作演示了将自己的 Schema USAGE 权限赋予其他用户，并收回 USAGE 权限。

步骤 1 首先在 finance 数据库创建 jim 用户：

```
create user jim identified by 'Bigdata@123';
```

截图如下：

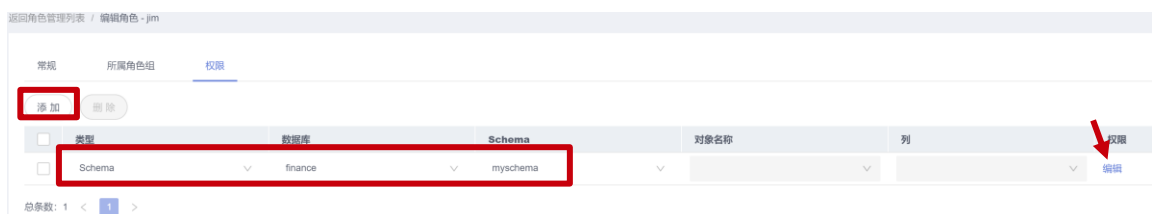


步骤 2 赋予 jim 用户 myschma 模式的 USAGE 权限

选择 账号管理 -> 角色管理，单击 jim 角色名，切换到 权限 界面：



单击“添加”，类型选择 Schema，数据库选择 finance，Schema 选择 myschma，单击“编辑”：



选择授予“USAGE”权限：

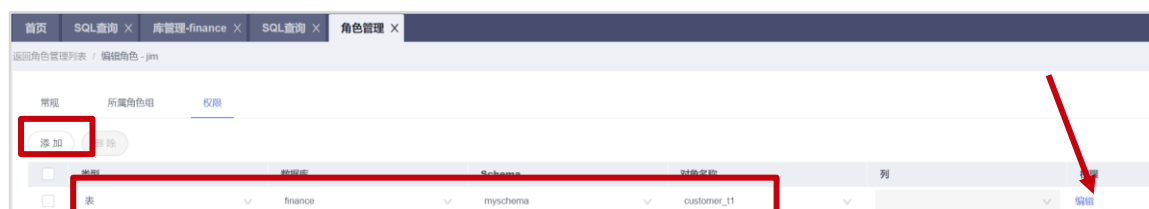


最后单击页面下方的“保存”后，查看 SQL 预览，单击“确定”后，赋予权限成功。



步骤 3 赋予 jim 用户 customer_t1 表的 SELECT 权限

单击 添加，类型选择表，数据库选择 finance，Schema 选择 myschema，对象名称选择 customer_t1，单击 编辑：



选择授予 SELECT 权限：



最后单击页面下方的“保存”后，查看 SQL 预览，单击确定后，赋予权限成功。

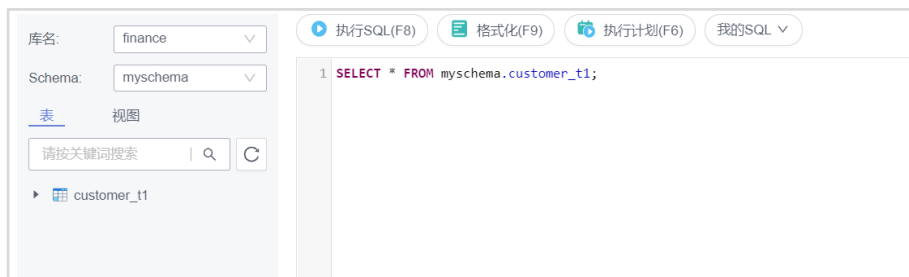
步骤 4 使用 jim 用户登录 finance 数据库，查看 myschema Schema 下的 customer_t1 表。



使用以下语句查看 customer_t1 表，查看成功：

```
SELECT * FROM myschema.customer_t1;
```

截图如下：



步骤 5 使用 root 用户登录 finance 数据库：

数据库登录

当前实例:

gauss-jx

gauss-jx - 192.168.0.207,192.168.0.110,192.168.0.50:8000

▼

当前最多展示100条数据库登录,可前往[数据库登录列表](#)查看更多。

* 数据库名称:

postgres

* 登录用户名:

root

密码:

×

☒ 记住密码

同意用户名及密码记录到DAS系统中,如不再需要,可以在[数据库登录列表](#)页面中删除。

☒ SQL执行记录 ②

开启此项后,您可以在DAS中,方便的查看到您的SQL窗口执行历史记录,并且可以直接再次执行,无需重复输入。

登录

步骤 6 将 jim 用户对于 myschema 的 USAGE 权限和 customer_t1 的 SELECT 权限收回。

选择“账号管理 -> 角色管理”，单击 jim 角色名，切换到“权限”界面，选择要收回的权限，然后单击删除：

常规 所属角色组 权限

添加 删除

☒

类型

☒

表

数据库	Schema	对象名称	列	权限
finance	myschema	customer_t1		SELECT

最后单击页面下方的“保存”后，查看 SQL 预览，单击“确定”后，完成权限收回。

1.4.1.2.2 删除 schema

步骤 1 删除名为 myschema 的 schema。

进入“库管理-finance”界面，选择 myschema 的操作“删除 Schema”：



Schema名称	表数量	表大小	索引大小	操作
dbesql_util	0	0B	0B	打开Schema 删除Schema
jim	0	0B	0B	打开Schema 删除Schema
myschema	1	0B	0B	打开Schema 删除Schema
public	0	0B	0B	打开Schema 删除Schema
root	0	0B	0B	打开Schema 删除Schema

单击 是，删除 Schema：



删除schema

确定删除schema myschema吗?
删除操作无法恢复，请谨慎操作。

Schema名称	表数量	表大小	索引大小
myschema	--	--	--

是 否

删除成功：



Schema名称	表数量	表大小	索引大小	操作
dbesql_util	0	0B	0B	打开Schema 删除Schema
jim	0	0B	0B	打开Schema 删除Schema
public	0	0B	0B	打开Schema 删除Schema
root	0	0B	0B	打开Schema 删除Schema

步骤 2 删除 jim 用户：

在 SQL 查询窗口执行如下语句：

```
DROP USER jim CASCADE;
```



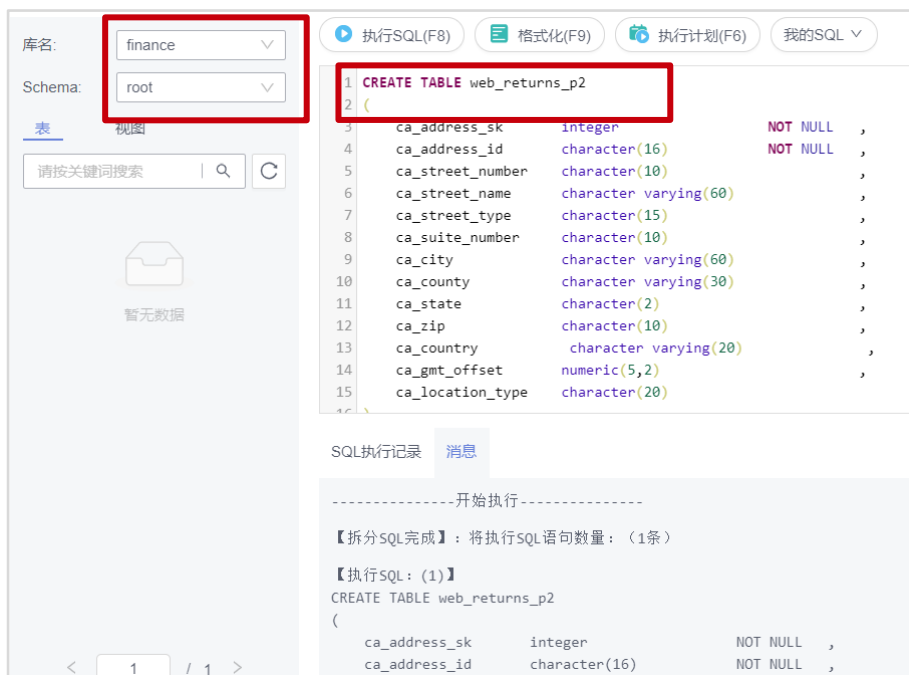
1.4.2 创建和管理分区表

1.4.2.1 创建分区表

步骤 1 打开 SQL 查询窗口，切换到 finance 数据库，root Schema，输入以下语句创建分区表。

```
CREATE TABLE web_returns_p2
(
    ca_address_sk      integer          NOT NULL ,
    ca_address_id      character(16)    NOT NULL ,
    ca_street_number   character(10)    ,
    ca_street_name     character varying(60) ,
    ca_street_type     character(15)    ,
    ca_suite_number    character(10)    ,
    ca_city            character varying(60) ,
    ca_county          character varying(30) ,
    ca_state           character(2)     ,
    ca_zip             character(10)    ,
    ca_country         character varying(20) ,
    ca_gmt_offset      numeric(5,2)    ,
    ca_location_type   character(20)    ,
)
PARTITION BY RANGE (ca_address_sk)
(
    PARTITION P1 VALUES LESS THAN(5000),
    PARTITION P2 VALUES LESS THAN(10000),
    PARTITION P3 VALUES LESS THAN(15000),
    PARTITION P4 VALUES LESS THAN(20000),
    PARTITION P5 VALUES LESS THAN(25000),
    PARTITION P6 VALUES LESS THAN(30000),
    PARTITION P7 VALUES LESS THAN(40000),
    PARTITION P8 VALUES LESS THAN(MAXVALUE)
)
ENABLE ROW MOVEMENT;
```

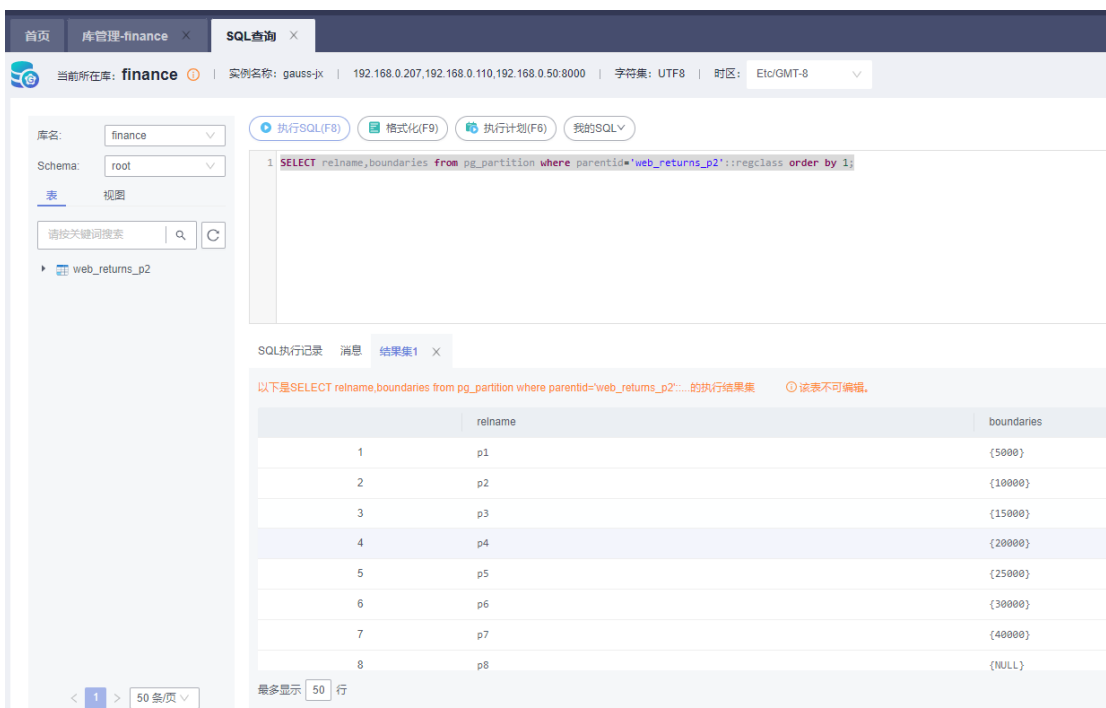
截图如下：



步骤 2 查询分区表的分区情况：

SELECT relname,boundaries from pg_partition where parentid='web_returns_p2'::regclass order by 1;

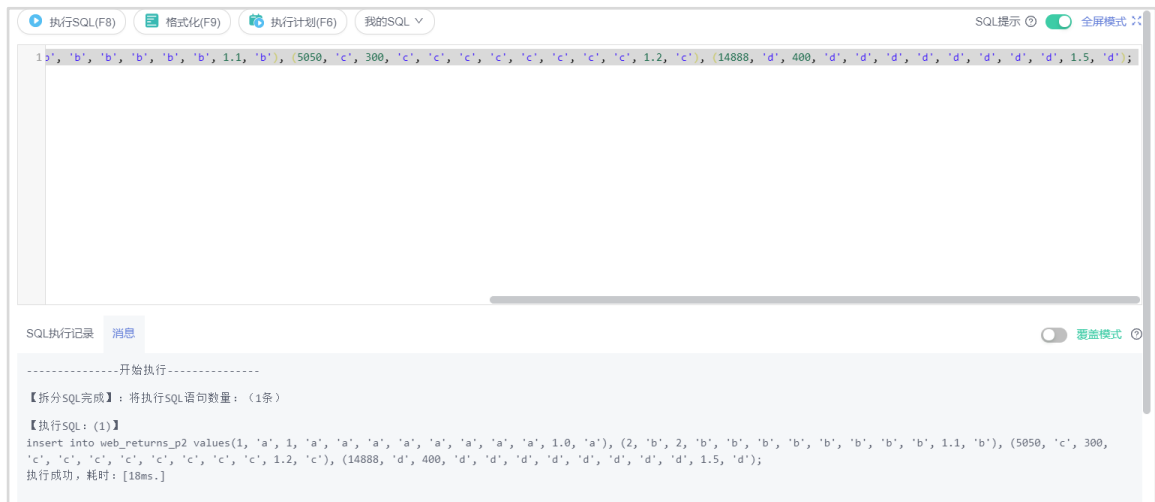
显示如下：



步骤 3 输入以下语句，插入数据。

insert into web_returns_p2 values(1, 'a', 1, 'a', 'a', 'a', 'a', 'a', 'a', 'a', 'a', 1.0, 'a'), (2, 'b', 2, 'b', 'b', 'b', 'b', 'b', 'b', 'b', 'b', 1.1, 'b'), (5050, 'c', 300, 'c', 'c', 'c', 'c', 'c', 'c', 'c', 'c', 1.2, 'c'), (14888, 'd', 400, 'd', 'd', 'd', 'd', 'd', 'd', 'd', 'd', 1.5, 'd');

截图如下：



1.4.2.2 管理分区表

1.4.2.2.1 修改分区表行迁移属性

输入以下语句，完成修改分区表行迁移属性：

```
ALTER TABLE web_returns_p2 DISABLE ROW MOVEMENT;
```



1.4.2.2.2 删除分区

输入以下语句，完成删除分区 P8。

```
ALTER TABLE web_returns_p2 DROP PARTITION P8;
```

截图如下：



1.4.2.2.3 增加分区

输入以下语句，增加分区 P8，范围为 40000<= P8<=MAXVALUE。

```
ALTER TABLE web_returns_p2 ADD PARTITION P8 VALUES LESS THAN (MAXVALUE);
```

截图如下：



1.4.2.2.4 重命名分区

输入以下语句，重命名分区 P8 为 P_9。

```
ALTER TABLE web_returns_p2 RENAME PARTITION P8 TO P_9;
```

截图如下：



输入以下语句，重命名分区 P_9 为 P8。

```
ALTER TABLE web_returns_p2 RENAME PARTITION FOR (40000) TO P8;
```

截图如下：



1.4.2.2.5 查询分区

输入以下语句，查询分区信息。

```
SELECT relname,boundaries from pg_partition where parentid='web_returns_p2'::regclass order by 1;
```

截图如下：

库名: finance Schema: root

SQL 执行记录 消息 结果集1 X

以下是SELECT reaname,boundaries from pg_partition where parentid='web_returns_p2'::regclass order by 1;

	reaname	boundaries
1	p1	{5000}
2	p2	{10000}
3	p3	{15000}
4	p4	{20000}
5	p5	{25000}
6	p6	{30000}
7	p7	{40000}
8	p8	{NULL}
9	web_returns_p2	

1.4.2.2.6 查询分区数据

输入以下语句，查询分区 P1。

```
SELECT * FROM web_returns_p2 PARTITION (P1);
```

截图如下：

SQL 执行记录 消息 结果集1 X

以下是SELECT * FROM web_returns_p2 PARTITION (P1);的执行结果集

	ca_address_sk	ca_address_id	ca_street_number	ca_street_name	ca_street_type	ca
1	1	a	1	a	a	a
2	2	b	2	b	b	b

1.4.2.2.7 删除分区表和表空间

输入以下语句：

```
DROP TABLE web_returns_p2;
```

截图如下：



1.4.3 创建和管理索引

索引可以提高数据的访问速度，但同时也增加了插入、更新和删除表的处理时间。所以是否要为表增加索引，索引建立在哪些字段上，是创建索引前必须要考虑的问题。需要分析应用程序的业务处理、数据使用、经常被用作查询条件或者被要求排序的字段来确定是否建立索引。

1.4.3.1 准备工作

步骤 1 输入以下语句，创建表并在表中插入数据。

```
CREATE TABLE web_returns_p2
(
    ca_address_sk      integer          NOT NULL ,
    ca_address_id      character(16)    NOT NULL ,
    ca_street_number   character(10)    ,
    ca_street_name     character varying(60) ,
    ca_street_type     character(15)    ,
    ca_suite_number    character(10)    ,
    ca_city            character varying(60) ,
    ca_county          character varying(30) ,
    ca_state           character(2)     ,
    ca_zip             character(10)    ,
    ca_country         character varying(20) ,
    ca_gmt_offset      numeric(5,2)    ,
    ca_location_type   character(20)    ,
)
PARTITION BY RANGE (ca_address_sk)
(
    PARTITION P1 VALUES LESS THAN(5000),
    PARTITION P2 VALUES LESS THAN(10000),
    PARTITION P3 VALUES LESS THAN(15000),
    PARTITION P4 VALUES LESS THAN(20000),
)
```

```

PARTITION P5 VALUES LESS THAN(25000),
PARTITION P6 VALUES LESS THAN(30000),
PARTITION P7 VALUES LESS THAN(40000),
PARTITION P8 VALUES LESS THAN(MAXVALUE)
)
ENABLE ROW MOVEMENT;

```

截图如下：

执行SQL(F8)
格式化(F9)
执行计划(F6)
我的SQL v

```

1 CREATE TABLE web_returns_p2
2 (
3     ca_address_sk      integer          NOT NULL ,
4     ca_address_id      character(16)    NOT NULL ,
5     ca_street_number   character(10)    ,
6     ca_street_name     character varying(60) ,
7     ca_street_type     character(15)    ,
8     ca_suite_number    character(10)    ,
9     ca_city            character varying(60) ,
10    ca_county           character varying(30)

```

SQL执行记录
消息

-----开始执行-----

【拆分SQL完成】：将执行SQL语句数量：（1条）

【执行SQL：（1）】

```

CREATE TABLE web_returns_p2
(
    ca_address_sk      integer          NOT NULL ,
    ca_address_id      character(16)    NOT NULL ,
    ca_street_number   character(10)    ,
    ca_street_name     character varying(60) ,
    ca_street_type     character(15)    ,

```

步骤 2 输入以下语句，插入数据。

```

insert into web_returns_p2 values(1, 'a', 1, 'a', 'a', 'a', 'a', 'a', 'a', 'a', 'a', 1.0, 'a'), (2, 'b', 2, 'b', 'b', 'b', 'b', 'b', 'b', 'b', 'b', 1.1, 'b'), (5050, 'c', 300, 'c', 'c', 'c', 'c', 'c', 'c', 'c', 'c', 1.2, 'c'), (14888, 'd', 400, 'd', 'd', 'd', 'd', 'd', 'd', 'd', 'd', 1.5, 'd');

```

截图如下：

执行SQL(F8)
格式化(F9)
执行计划(F6)
我的SQL v
SQL提示 ⓘ 全屏模式 ⌵

```

1 'b', 'b', 'b', 1.1, 'b'), (5050, 'c', 300, 'c', 'c', 'c', 'c', 'c', 'c', 'c', 'c', 1.2, 'c'), (14888, 'd', 400, 'd', 'd', 'd', 'd', 'd', 'd', 'd', 'd', 1.5, 'd');

```

SQL执行记录
消息
覆盖模式 ⓘ

-----开始执行-----

【拆分SQL完成】：将执行SQL语句数量：（1条）

【执行SQL：（1）】

```

insert into web_returns_p2 values(1, 'a', 1, 'a', 'a', 'a', 'a', 'a', 'a', 'a', 'a', 1.0, 'a'), (2, 'b', 2, 'b', 'b', 'b', 'b', 'b', 'b', 'b', 'b', 1.1, 'b'), (5050, 'c', 300, 'c', 'c', 'c', 'c', 'c', 'c', 'c', 'c', 1.2, 'c'), (14888, 'd', 400, 'd', 'd', 'd', 'd', 'd', 'd', 'd', 'd', 1.5, 'd');
执行成功，耗时：[17ms.]

```

1.4.3.2 创建索引

输入以下语句，创建分区表索引 tpcds_web_returns_p2_index1，不指定索引分区的名称。

```
CREATE INDEX tpcds_web_returns_p2_index1 ON web_returns_p2 (ca_address_id) LOCAL;
```

截图如下：



输入以下语句，创建分区索引 tpcds_web_returns_p2_index2，并指定索引分区的名称。

```
CREATE INDEX tpcds_web_returns_p2_index2 ON web_returns_p2 (ca_address_sk) LOCAL
(
    PARTITION web_returns_p2_P1_index,
    PARTITION web_returns_p2_P2_index,
    PARTITION web_returns_p2_P3_index,
    PARTITION web_returns_p2_P4_index,
    PARTITION web_returns_p2_P5_index,
    PARTITION web_returns_p2_P6_index,
    PARTITION web_returns_p2_P7_index,
    PARTITION web_returns_p2_P8_index
)
```

截图如下：



```

1 CREATE INDEX tpcds_web_returns_p2_index2 ON web_returns_p2 (ca_address_sk) LOCAL
2 (
3     PARTITION web_returns_p2_P1_index,
4     PARTITION web_returns_p2_P2_index,
5     PARTITION web_returns_p2_P3_index,
6     PARTITION web_returns_p2_P4_index,
7     PARTITION web_returns_p2_P5_index,
8     PARTITION web_returns_p2_P6_index,
9     PARTITION web_returns_p2_P7_index.

```

SQL执行记录 消息

-----开始执行-----

【拆分SQL完成】：将执行SQL语句数量：（1条）

【执行SQL：（1）】

CREATE INDEX tpcds_web_returns_p2_index2 ON web_returns_p2 (ca_address_sk) LOCAL
 (
 PARTITION web_returns_p2_P1_index,
 PARTITION web_returns_p2_P2_index,
 PARTITION web_returns_p2_P3_index,
 PARTITION web_returns_p2_P4_index,

1.4.3.3 管理索引

1.4.3.3.1 重命名索引分区

执行如下命令对索引分区 web_returns_p2_P8_index 重命名 web_returns_p2_P8_index_new。

```
ALTER INDEX tpcds_web_returns_p2_index2 RENAME PARTITION web_returns_p2_P8_index TO web_returns_p2_P8_index_new;
```

截图如下：



```

1 ALTER INDEX tpcds_web_returns_p2_index2 RENAME PARTITION web_returns_p2_P8_index TO web_returns_p2_P8_index_new;

```

SQL执行记录 消息

-----开始执行-----

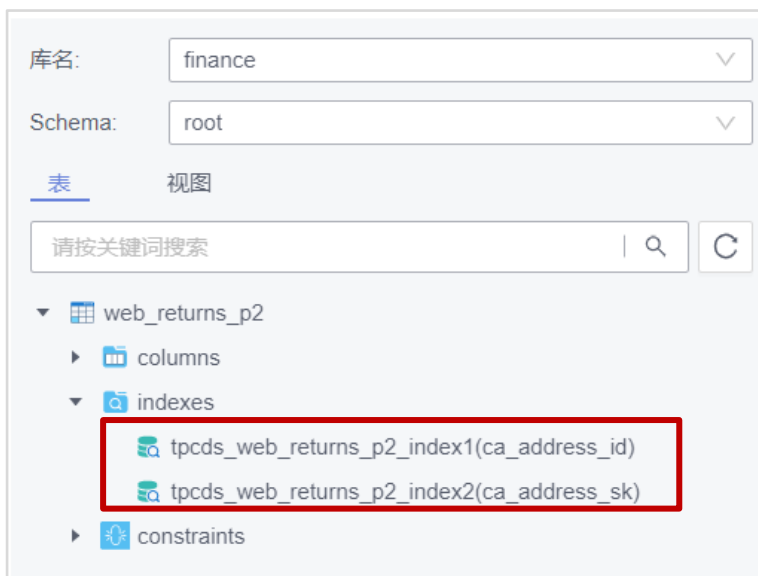
【拆分SQL完成】：将执行SQL语句数量：（1条）

【执行SQL：（1）】

ALTER INDEX tpcds_web_returns_p2_index2 RENAME PARTITION web_returns_p2_P8_index TO web_returns_p2_P8_index_new;
 执行成功，耗时：[20ms.]

1.4.3.3.2 查询索引

创建索引后刷新页面，左下角会显示表视图，单击 indexes 显示当前表的所有索引：



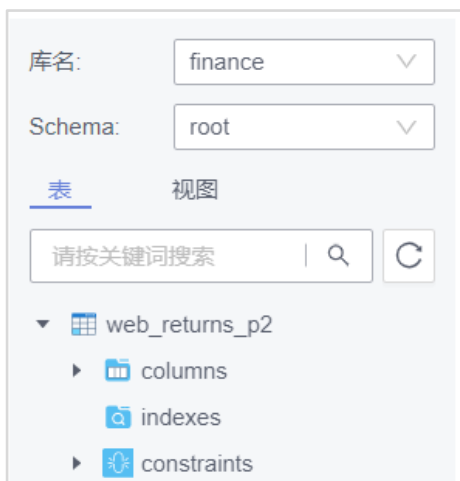
1.4.3.3.3 删除索引

输入以下语句，删除索引：

```
DROP INDEX tpcds_web_returns_p2_index1;
DROP INDEX tpcds_web_returns_p2_index2;
```

截图如下：





1.4.3.4 索引创建举例

GaussDB 支持 4 种创建索引的方式：唯一索引、多字段索引、部分索引、表达式索引。

步骤 1 输入以下语句，创建一个普通表。

```
CREATE TABLE customer_address_bak AS TABLE web_returns_p2;
```

截图如下：



步骤 2 创建普通索引。

如果对于 customer_address_bak 表，需要经常进行以下查询。

```
SELECT ca_address_sk FROM customer_address_bak WHERE ca_address_sk=14888;
```

截图如下：



使用以下命令创建索引。

```
CREATE INDEX index_wr_returned_date_sk ON customer_address_bak (ca_address_sk) LOCAL;
```

截图如下：



步骤 3 创建多字段索引

假如用户需要经常查询表 tpcds.customer_address_bak 中 ca_address_sk 是 5050，且 ca_street_number 小于 1000 的记录，使用以下命令进行查询。

```
SELECT ca_address_sk,ca_address_id FROM customer_address_bak WHERE ca_address_sk = 5050 AND ca_street_number < 1000;
```

截图如下：



使用以下命令在字段 ca_address_sk 和 ca_street_number 上定义一个多字段索引。

```
CREATE INDEX more_column_index ON customer_address_bak(ca_address_sk ,ca_street_number) LOCAL;
```

截图如下：



步骤 4 创建部分索引

如果只需要查询 ca_address_sk 为 5050 的记录，可以创建部分索引来提升查询效率。

```
CREATE INDEX part_index ON customer_address_bak(ca_address_sk) WHERE ca_address_sk = 5050;
```

截图如下：



步骤 5 创建表达式索引

假如经常需要查询 ca_street_number 小于 1000 的信息，执行如下命令进行查询。

```
SELECT * FROM customer_address_bak WHERE trunc(ca_street_number) < 1000;
```

截图如下：

SQL 提示 ⓘ 全屏模式 ✕

1 `SELECT * FROM customer_address_bak WHERE trunc(ca_street_number) < 1000;`

SQL 执行记录 消息 结果集1 X 覆盖模式 ⓘ

以下是SELECT * FROM customer_address_bak WHERE trunc(ca_street_number) < 1000的执行结果集 该表不可编辑。 复制行 复制列 列设置

	ca_address_sk	ca_address_id	ca_street_number	ca_street_name	ca_street_type	ca_suite_number
1	1	a	1	a	a	a
2	2	b	2	b	b	b
3	5050	c	300	c	c	c
4	14888	d	400	d	d	d

可以为上面的查询创建表达式索引：

```
CREATE INDEX para_index ON customer_address_bak (trunc(ca_street_number)) LOCAL;
```

截图如下：

执行SQL(F8) 格式化(F9) 执行计划(F6) 我的SQL v

1 `CREATE INDEX para_index ON customer_address_bak (trunc(ca_street_number));`

SQL 执行记录 消息

-----开始执行-----

【拆分SQL完成】：将执行SQL语句数量：（1条）

【执行SQL：（1）】

CREATE INDEX para_index ON customer_address_bak (trunc(ca_street_number));

执行成功，耗时：[29ms.]

步骤 6 删除 customer_address_bak 表。

```
DROP TABLE customer_address_bak;
```

截图如下：



1.4.4 创建和管理视图

- 当用户对数据库中的一张或者多张表的某些字段的组合感兴趣，而又不想每次键入这些查询时，用户就可以定义一个视图，以便解决这个问题。
- 视图与基本表不同，不是物理上实际存在的，是一个虚表。数据库中仅存放视图的定义，而不存放视图对应的数据，这些数据仍存放在原来的基本表中。若基本表中的数据发生变化，从视图中查询出的数据也随之改变。从这个意义上讲，视图就像一个窗口，透过它可以看到数据库中用户感兴趣的数据及变化。视图每次被引用的时候都会运行一次。
- 物化视图是一种特殊的物理表，物化视图是相对普通视图而言的。普通视图是虚拟表，应用的局限性较大，任何对视图的查询实际上都是转换为对 SQL 语句的查询，性能并没有实际上提高。物化视图实际上就是存储 SQL 执行语句的结果，起到缓存的效果。

1.4.4.1 创建视图

执行如下命令创建普通视图 MyView。

```
CREATE OR REPLACE VIEW MyView AS SELECT * FROM web_returns_p2 WHERE trunc(ca_address_sk) > 10000;
```

截图如下：



执行如下命令创建物化视图 MV_MyView。

```
CREATE MATERIALIZED VIEW MV_MyView AS SELECT * FROM web_returns_p2 WHERE
trunc(ca_address_sk) > 5000;
```

截图如下：



注：

- 物化视图使用场景：报表统计、大表统计等，定期固化数据快照，避免对多表重复跑相同的查询。
- 物化视图使用注意事项：
 - 不可以临时表或全局临时表上创建。
 - 当基表数据发生变化时，需要使用刷新命令保持物化视图与基表同步。

1.4.4.2 管理视图

1.4.4.2.1 查询普通视图

执行如下命令查询 MyView 视图。

```
SELECT * FROM MyView;
```

截图如下：



1.4.4.2.2 查看普通视图的具体信息

步骤 1 切换到“库管理 -> 对象列表”，单击“视图”，查看视图列表，选中 myview 视图的操作，单击“查看视图详情”：



步骤 2 查看视图详情



1.4.4.2.3 查询物化视图

执行如下命令查询 MV_MyView 视图。

```
SELECT * FROM MV_MyView;
```

截图如下：

	ca_address_sk	ca_address_id	ca_street_number	ca_street_name	ca_street_type	ca_suite_number
1	5050	c	300	c	c	c
2	14888	d	400	d	d	d

1.4.4.2.4 刷新物化图

步骤 1 给基表新增二条记录，然后刷新物化视图。

```
insert into web_returns_p2 values (7050, 'c', 300, 'c', 'c', 'c', 'c', 'c', 'c', 'c', 'c', 1.2, 'c'), (8888, 'd', 400, 'd', 'd', 'd', 'd', 'd', 'd', 'd', 'd', 1.5, 'd');
```

截图如下：

```
insert into web_returns_p2 values (7050, 'c', 300, 'c', 'c', 'c', 'c', 'c', 'c', 'c', 'c', 1.2, 'c'), (8888, 'd', 400, 'd', 'd', 'd', 'd', 'd', 'd', 'd', 'd', 1.5, 'd');
```

【拆分SQL完成】：将执行SQL语句数量：（1条）

【执行SQL：（1）】

insert into web_returns_p2 values (7050, 'c', 300, 'c', 'c', 'c', 'c', 'c', 'c', 'c', 'c', 1.2, 'c'), (8888, 'd', 400, 'd', 'd', 'd', 'd', 'd', 'd', 'd', 'd', 1.5, 'd');

执行成功，耗时：[10ms.]

步骤 2 由于基表数据变更过，可以执行如下命令刷新物化视图 MV_MyView。

```
REFRESH MATERIALIZED VIEW MV_MyView;
```

截图如下：

```
REFRESH MATERIALIZED VIEW MV_MyView;
```

【拆分SQL完成】：将执行SQL语句数量：（1条）

【执行SQL：（1）】

REFRESH MATERIALIZED VIEW MV_MyView;

执行成功，耗时：[45ms.]

步骤 3 再查看物化视图 MV_MyView，发现多了二条记录。

```
SELECT * FROM MV_MyView;
```

截图如下：



SQL 执行记录 消息 结果集1 X

以下是SELECT * FROM MV_MyView的执行结果集 该表不可编辑。

	ca_address_sk	ca_address_id	ca_street_number	ca_street_name	ca_street_type	ca_suite_number
1	5050	c	300	c	c	c
2	7050	c	300	c	c	c
3	8888	d	400	d	d	d
4	14888	d	400	d	d	d

1.4.4.2.5 删除视图

执行如下命令删除视图。

```
DROP VIEW MyView;
DROP MATERIALIZED VIEW MV_MyView;
```

截图如下：



SQL 执行记录 消息

-----开始执行-----

【拆分SQL完成】：将执行SQL语句数量：（2条）

【执行SQL：（1）】
DROP VIEW MyView;
执行成功，耗时：[7ms.]

【执行SQL：（2）】
DROP MATERIALIZED VIEW MV_MyView;
执行成功，耗时：[44ms.]

1.4.5 创建和管理序列

1.4.5.1 方法一：声明字段类型为序列整型(serial)来定义标识符字段。

例如：

```
CREATE TABLE T1
(id serial,
name text
);
```

截图如下：



1.4.5.2 方法二：创建序列，并通过 nextval('sequence_name')函数指定为某一字段的默认值。

步骤 1 创建序列。

```
CREATE SEQUENCE seq1 cache 100;
```

截图如下：



步骤 2 指定为某一字段的默认值，使该字段具有唯一标识属性。

```
CREATE TABLE T2
(
    id int not null default nextval('seq1'),
    name text
);
```

截图如下：



步骤 3 指定序列与列的归属关系。

将序列和一个表的指定字段进行关联。删除此字段或其所在表的时候会自动删除已关联的序列。

```
ALTER SEQUENCE seq1 OWNED BY T2.id;
```

截图如下：



1.4.5.3 删除序列

```
DROP SEQUENCE seq1 CASCADE;
```

截图如下：



1.4.6 创建和管理存储过程

1.4.6.1 创建存储过程

步骤 1 创建表 t_test。

```
create table t_test(c1 int, c2 int);
```

截图如下：



步骤 2 创建存储过程 insert_data。

```
create or replace procedure insert_data
is
a int;
```

```
b int;
begin
a=1;
b=2;
insert into t_test values(a,b);
insert into t_test values(b,a);
end;
/
```

截图如下：



The screenshot shows the GaussDB DAS interface with the following components:

- Top Bar:** Contains buttons for "执行SQL(F8)", "格式化(F9)", "执行计划(F6)", and a dropdown for "我的SQL".
- SQL Editor:** Displays the PL/SQL code for creating and replacing the procedure `insert_data`. The code is as follows:

```
1 create or replace procedure insert_data
2 is
3 a int;
4 b int;
5 begin
6 a=1;
7 b=2;
```
- SQL Execution Record:** Shows the execution status and details.
 - Start Execution:** Indicated by a dashed line.
 - SQL Splitting:** A message states "【拆分SQL完成】：将执行SQL语句数量：（1条）".
 - Execution SQL:** A message states "【执行SQL：（1）】".
 - Execution Details:** The full PL/SQL code is displayed again, followed by the message "执行成功，耗时：[15ms.]".

步骤 3 调用存储过程。

```
call insert_data();
```

截图如下：



The screenshot shows the GaussDB DAS interface with the following components:

- Top Bar:** Contains buttons for "执行SQL(F8)", "格式化(F9)", "执行计划(F6)", and a dropdown for "我的SQL".
- SQL Editor:** Displays the call statement:

```
1 call insert_data();
2
3
4
```
- SQL Execution Record:** Shows the execution status and details.
 - Execution Result:** A message states "以下是call insert_data(),的执行结果集".
 - Table Name:** The table `insert_data` is listed.
 - Execution Result:** A single row is displayed with the value `1`.
 - Message:** A message states "该表不可编辑。".

步骤 4 查询表内容。

```
select * from t_test;
```

截图如下：

	c1	c2
1	1	2
2	2	1

1.4.6.2 管理存储过程

管理存储过程，切换到“库管理 -> 对象列表”，选择“存储过程”，选择 insert_data 存储过程中的操作，单击查看存储过程详情：

存储过程名称	参数	操作
insert_data	()	修改或执行 删除存储过程 查看存储过程详情

查看存储过程详情：



切换到 SQL 查询界面，删除存储过程，命令如下：

```
drop procedure insert_data;
```

截图如下：



1.4.7 创建和管理全局临时表

1.4.7.1 会话级全局临时表

数据会话级可见，其他会话看不到数据，但表结构可见。

步骤 1 创建临时表 t_test2。

建表语句，使用 ON COMMIT PRESERVE ROWS

```
CREATE GLOBAL TEMPORARY TABLE t_test2(
id integer,
lbl text
```

) ON COMMIT PRESERVE ROWS;

截图如下：



步骤 2 在当前会话插入数据并查询。

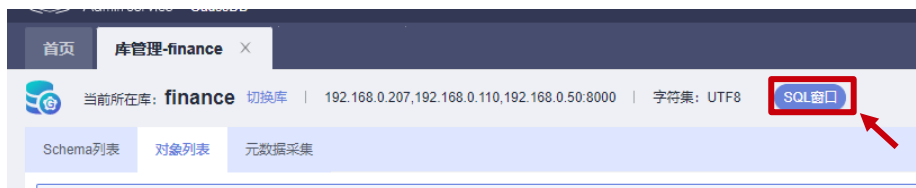
insert into t_test2 values(1,'data1');
insert into t_test2 values(2,'data2');
select * from t_test2;

截图如下：



步骤 3 关闭当前 SQL 查询窗口，通过单击“库管理-finance”页面中的“SQL 窗口”重新打开一个“SQL 查询”窗口。





步骤 4 查询表内容。

```
select * from t_test2;
```

截图如下：



显示表已经不存在了。

1.4.7.2 事务级全局临时表

数据事务级可见，事务提交后数据删除。

步骤 1 创建临时表 t_test3。

建表语句，使用 ON COMMIT DELETE ROWS：

```
CREATE GLOBAL TEMPORARY TABLE t_test3(
id integer,
lbl text
) ON COMMIT DELETE ROWS;
```

截图如下：

执行SQL(F8)
格式化(F9)
执行计划(F6)
我的SQL

```

1 CREATE GLOBAL TEMPORARY TABLE t_test3(
2 id integer,
3 lbl text
4 ) ON COMMIT DELETE ROWS;
5

```

SQL执行记录
消息

```

-----开始执行-----

【拆分SQL完成】：将执行SQL语句数量：（1条）

【执行SQL：（1）】
CREATE GLOBAL TEMPORARY TABLE t_test3(
id integer,
lbl text
) ON COMMIT DELETE ROWS;
执行成功，耗时：[9ms.]

```

步骤 2 插入数据并查询。

先用 begin 开始一个事务，接着给表插入数据，此时再对表进行查询，可以查出相应数据。

```

begin;
insert into t_test3 values(1,'data1');
select * from t_test3;

```

截图如下：

执行SQL(F8)
格式化(F9)
执行计划(F6)
我的SQL

SQL提示 全屏模式

```

1 begin;
2 insert into t_test3 values(1,'data1');
3 select * from t_test3;
4
5

```

SQL执行记录
消息
结果集1 X

覆盖模式

以下是select * from t_test3;的执行结果集 该表不可编辑。

	id	lbl
1	1	data1

复制行
复制列
列设置

步骤 3 结束事务再查询。

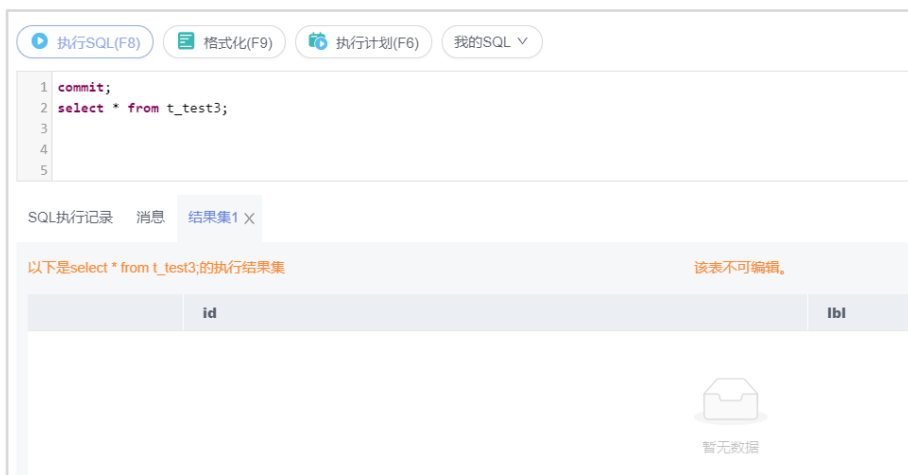
先用 commit 提交来结束事务，此时再对表进行查询，可以发现已经查询不出数据了。

```

commit;
select * from t_test3;

```

截图如下：



步骤 4 删除临时表。

drop table t_test3;

截图如下：



1.5 学校数据模型

1.5.1 关于本实验

以学校数据库模型为例，介绍 GaussDB 数据库、表、表空间、用户及其它对象，以及 SQL 语法使用的介绍。

假设 A 市 B 学校为了加强对学校的管理，引入了华为 GaussDB 数据库。在 B 学校里，主要涉及的对象有学生、教师、班级、院系和课程。本实验假设在 B 学校数据库中，教师会教授课程，学生会选修课程，院系会聘请教师，班级会组成院系，学生会组成班级。因此，根据此关系，本实验给出了相应的关系模式和 ER 图，并对其进行基本的数据库操作。

1.5.2 关系模型

对于 B 校中的 5 个对象，分别建立属于每个对象的属性集合，具体属性描述如下：

- 学生（学号，姓名，性别，出生日期，入学日期，家庭住址）
- 教师（教师编号，教师姓名，职称，性别，年龄，入职日期）
- 班级（班级编号，班级名称，班主任）
- 院系（系编号，系名称，系主任）
- 课程（课程编号，课程名称，课程类型，学分）

上述属性对应的编号为：

- student (std_id, std_name, std_sex, std_birth, std_in, std_address)
- teacher (tec_id, tec_name, tec_job, tec_sex, tec_age, tec_in)
- class (cla_id, cla_name, cla_teacher)
- school_department (depart_id, depart_name, depart_teacher)
- course (cor_id, cor_name, cor_type, credit)

对象之间的关系：

- 一位学生可以选择多门课程，一门课程可被多名学生选择
- 一位老师可以选择多门课程，一门课程可被多名老师教授
- 一个院系可由多个班级组成
- 一个院系可聘请多名老师
- 一个班级可由多名学生组成

1.5.3 E-R 图

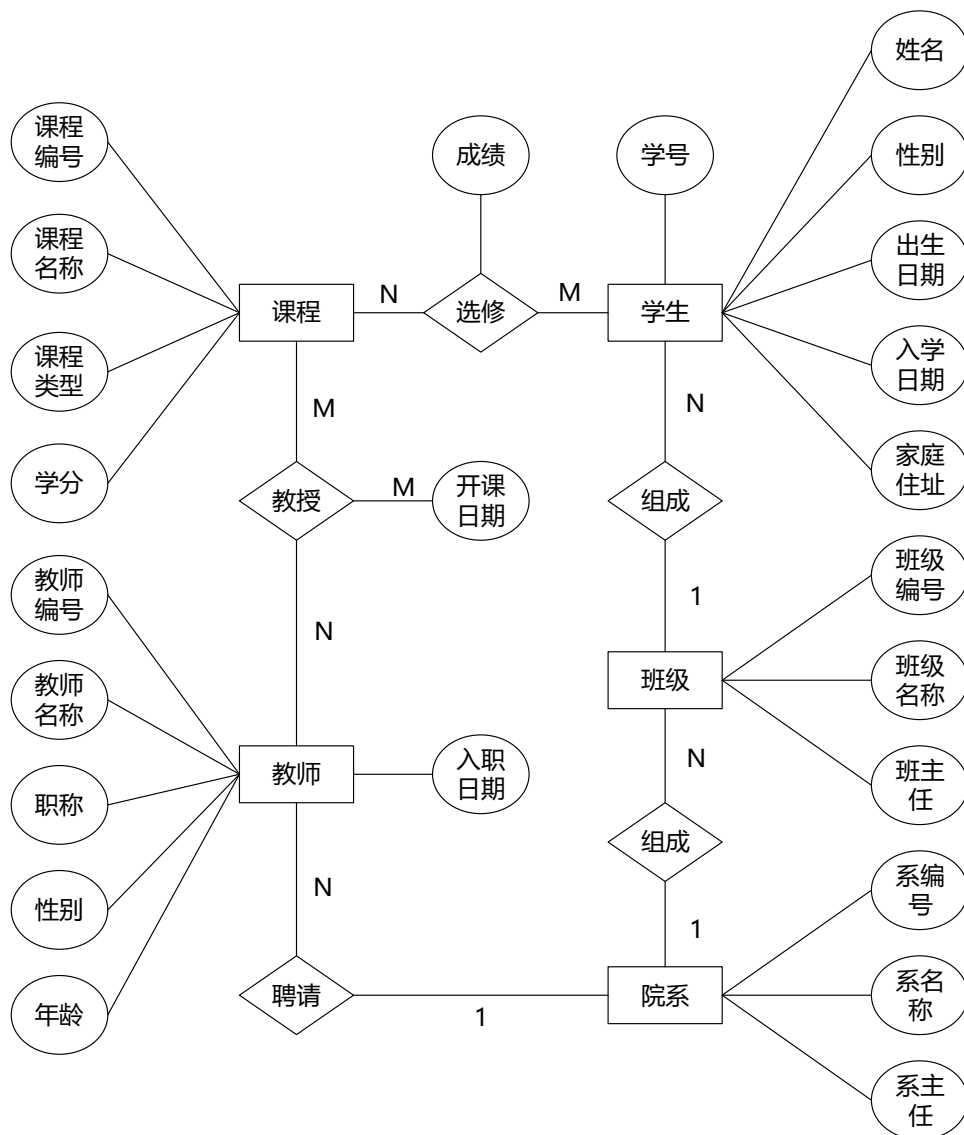


图1-1 E-R 图

1.6 学校数据模型表操作

1.6.1 表的创建

根据 B 学校的场景描述，本实验分别针对学生（student），教师（teacher），班级（class），院系（school_department）和课程（course）创建相应的表。具体的实验步骤如下所示：

步骤 1 创建学生信息表。

DROP TABLE IF EXISTS student;

截图如下：



创建 student 表：

```
CREATE TABLE student
(
    std_id INT PRIMARY KEY,
    std_name NCHAR(20) NOT NULL,
    std_sex NCHAR(6),
    std_birth DATE,
    std_in DATE NOT NULL,
    std_address VARCHAR(100)
);
```

截图如下：

执行SQL(F8)

格式化(F9)

执行计划(F6)

我的SQL v

```

1 CREATE TABLE student
2 (
3     std_id INT PRIMARY KEY,
4     std_name NCHAR(20) NOT NULL,
5     std_sex NCHAR(6),
6     std_birth DATE,
7     std_in DATE NOT NULL,
8     std_address VARCHAR(100)
9 );
10

```

SQL执行记录

消息

-----开始执行-----

【拆分SQL完成】：将执行SQL语句数量：（1条）

【执行SQL：（1）】

```

CREATE TABLE student
(
    std_id INT PRIMARY KEY,
    std_name NCHAR(20) NOT NULL,
    std_sex NCHAR(6),
    std_birth DATE,
    std_in DATE NOT NULL,
    std_address VARCHAR(100)
);
执行成功，耗时：[37ms.]

```

warning:

CREATE TABLE / PRIMARY KEY will create implicit index "student_pkey" for table "student"

步骤 2 创建教师信息表。

```

DROP TABLE IF EXISTS teacher;
CREATE TABLE teacher
(
    tec_id INT PRIMARY KEY,
    tec_name CHAR(20) NOT NULL,
    tec_job CHAR(15),
    tec_sex CHAR(6),
    tec_age INT,
    tec_in DATE NOT NULL
);

```

截图如下：

执行SQL(F8)

格式化(F9)

执行计划(F6)

我的SQL v

```

1 DROP TABLE IF EXISTS teacher;
2 CREATE TABLE teacher
3 (
4     tec_id INT PRIMARY KEY,
5     tec_name CHAR(20) NOT NULL,
6     tec_job CHAR(15),
7     tec_sex CHAR(6),
8     tec_age INT,
9     tec_in DATE NOT NULL
10 );
11

```

SQL执行记录

消息

warning:

table "teacher" does not exist, skipping

【执行SQL：（2）】

```

CREATE TABLE teacher
(
    tec_id INT PRIMARY KEY,
    tec_name CHAR(20) NOT NULL,
    tec_job CHAR(15),
    tec_sex CHAR(6),
    tec_age INT,
    tec_in DATE NOT NULL
);
执行成功，耗时：[26ms.]

```

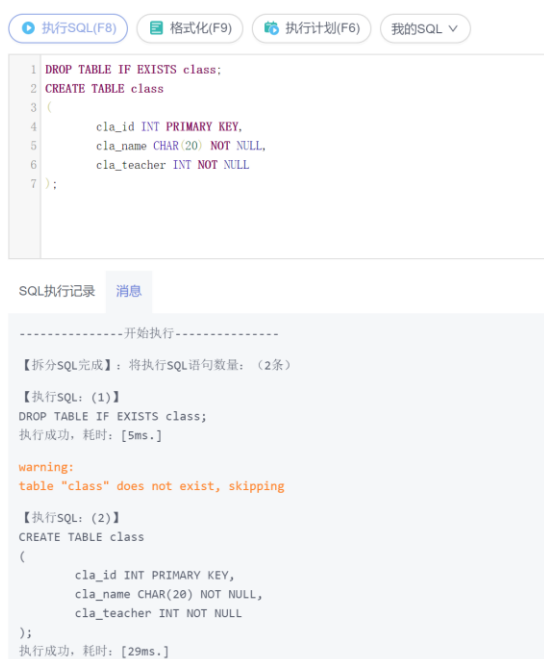
warning:

CREATE TABLE / PRIMARY KEY will create implicit index "teacher_pkey" for table "teacher"

步骤 3 创建班级信息表。

```
DROP TABLE IF EXISTS class;
CREATE TABLE class
(
    cla_id INT PRIMARY KEY,
    cla_name CHAR(20) NOT NULL,
    cla_teacher INT NOT NULL
);
```

截图如下：



步骤 4 创建院系信息表。

```
DROP TABLE IF EXISTS school_department;
CREATE TABLE school_department
(
    depart_id INT PRIMARY KEY,
    depart_name NCHAR(30) NOT NULL,
    depart_teacher INT NOT NULL
);
```

截图如下：

执行SQL(F8)
格式化(F9)
执行计划(F6)
我的SQL ▾

```

1 DROP TABLE IF EXISTS school_department;
2 CREATE TABLE school_department
3 (
4     depart_id INT PRIMARY KEY,
5     depart_name NCHAR(30) NOT NULL,
6     depart_teacher INT NOT NULL
7 );
8

```

SQL执行记录
消息

```

-----开始执行-----

【拆分SQL完成】：将执行SQL语句数量：（2条）

【执行SQL：（1）】
DROP TABLE IF EXISTS school_department;
执行成功，耗时：[5ms.]

warning:
table "school_department" does not exist, skipping

【执行SQL：（2）】
CREATE TABLE school_department
(
    depart_id INT PRIMARY KEY,
    depart_name NCHAR(30) NOT NULL,
    depart_teacher INT NOT NULL
);
执行成功，耗时：[37ms.]

```

步骤 5 创建课程信息表。

```

DROP TABLE IF EXISTS course;
CREATE TABLE course
(
    cor_id INT PRIMARY KEY,
    cor_name NCHAR(30) NOT NULL,
    cor_type NCHAR(20),
    credit numeric
);

```

截图如下：



1.6.2 表数据的插入

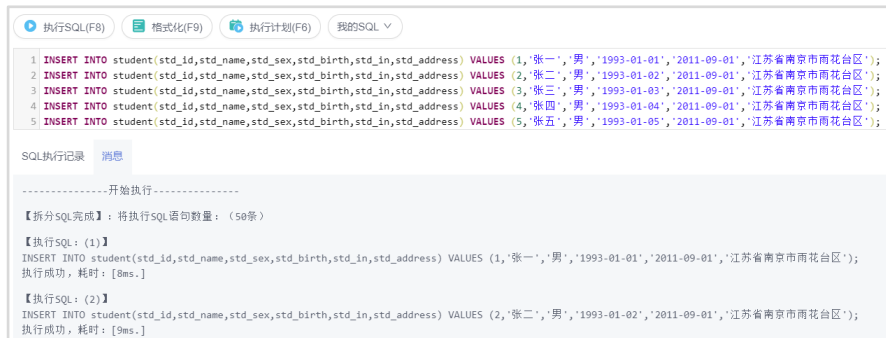
步骤 1 向 student 表中插入数据。

```
INSERT INTO student(std_id,std_name,std_sex,std_birth,std_in,std_address) VALUES (1,'张一','男','1993-01-01','2011-09-01','江苏省南京市雨花台区');
INSERT INTO student(std_id,std_name,std_sex,std_birth,std_in,std_address) VALUES (2,'张二','男','1993-01-02','2011-09-01','江苏省南京市雨花台区');
INSERT INTO student(std_id,std_name,std_sex,std_birth,std_in,std_address) VALUES (3,'张三','男','1993-01-03','2011-09-01','江苏省南京市雨花台区');
INSERT INTO student(std_id,std_name,std_sex,std_birth,std_in,std_address) VALUES (4,'张四','男','1993-01-04','2011-09-01','江苏省南京市雨花台区');
INSERT INTO student(std_id,std_name,std_sex,std_birth,std_in,std_address) VALUES (5,'张五','男','1993-01-05','2011-09-01','江苏省南京市雨花台区');
INSERT INTO student(std_id,std_name,std_sex,std_birth,std_in,std_address) VALUES (6,'张六','男','1993-01-06','2011-09-01','江苏省南京市雨花台区');
INSERT INTO student(std_id,std_name,std_sex,std_birth,std_in,std_address) VALUES (7,'张七','男','1993-01-07','2011-09-01','江苏省南京市雨花台区');
INSERT INTO student(std_id,std_name,std_sex,std_birth,std_in,std_address) VALUES (8,'张八','男','1993-01-08','2011-09-01','江苏省南京市雨花台区');
INSERT INTO student(std_id,std_name,std_sex,std_birth,std_in,std_address) VALUES (9,'张九','男','1993-01-09','2011-09-01','江苏省南京市雨花台区');
INSERT INTO student(std_id,std_name,std_sex,std_birth,std_in,std_address) VALUES (10,'李一','男','1993-01-10','2011-09-01','江苏省南京市雨花台区');
INSERT INTO student(std_id,std_name,std_sex,std_birth,std_in,std_address) VALUES (11,'李二','男','1993-01-11','2011-09-01','江苏省南京市雨花台区');
INSERT INTO student(std_id,std_name,std_sex,std_birth,std_in,std_address) VALUES (12,'李三','男','1993-01-12','2011-09-01','江苏省南京市雨花台区');
INSERT INTO student(std_id,std_name,std_sex,std_birth,std_in,std_address) VALUES (13,'李四','男','1993-01-13','2011-09-01','江苏省南京市雨花台区');
INSERT INTO student(std_id,std_name,std_sex,std_birth,std_in,std_address) VALUES (14,'李五','男','1993-01-14','2011-09-01','江苏省南京市雨花台区');
```

```
INSERT INTO student(std_id,std_name,std_sex,std_birth,std_in,std_address) VALUES (15,'李六','男','1993-01-15','2011-09-01','江苏省南京市雨花台区');
INSERT INTO student(std_id,std_name,std_sex,std_birth,std_in,std_address) VALUES (16,'李七','男','1993-01-16','2011-09-01','江苏省南京市雨花台区');
INSERT INTO student(std_id,std_name,std_sex,std_birth,std_in,std_address) VALUES (17,'李八','男','1993-01-17','2011-09-01','江苏省南京市雨花台区');
INSERT INTO student(std_id,std_name,std_sex,std_birth,std_in,std_address) VALUES (18,'李九','男','1993-01-18','2011-09-01','江苏省南京市雨花台区');
INSERT INTO student(std_id,std_name,std_sex,std_birth,std_in,std_address) VALUES (19,'王一','男','1993-01-19','2011-09-01','江苏省南京市雨花台区');
INSERT INTO student(std_id,std_name,std_sex,std_birth,std_in,std_address) VALUES (20,'王二','男','1993-01-20','2011-09-01','江苏省南京市雨花台区');
INSERT INTO student(std_id,std_name,std_sex,std_birth,std_in,std_address) VALUES (21,'王三','男','1993-01-21','2011-09-01','江苏省南京市雨花台区');
INSERT INTO student(std_id,std_name,std_sex,std_birth,std_in,std_address) VALUES (22,'王四','男','1993-01-22','2011-09-01','江苏省南京市雨花台区');
INSERT INTO student(std_id,std_name,std_sex,std_birth,std_in,std_address) VALUES (23,'王五','男','1993-01-23','2011-09-01','江苏省南京市雨花台区');
INSERT INTO student(std_id,std_name,std_sex,std_birth,std_in,std_address) VALUES (24,'王六','男','1993-01-24','2011-09-01','江苏省南京市雨花台区');
INSERT INTO student(std_id,std_name,std_sex,std_birth,std_in,std_address) VALUES (25,'王七','男','1993-01-25','2011-09-01','江苏省南京市雨花台区');
INSERT INTO student(std_id,std_name,std_sex,std_birth,std_in,std_address) VALUES (26,'王八','男','1993-01-26','2011-09-01','江苏省南京市雨花台区');
INSERT INTO student(std_id,std_name,std_sex,std_birth,std_in,std_address) VALUES (27,'王九','男','1993-01-27','2011-09-01','江苏省南京市雨花台区');
INSERT INTO student(std_id,std_name,std_sex,std_birth,std_in,std_address) VALUES (28,'钱一','男','1993-01-28','2011-09-01','江苏省南京市雨花台区');
INSERT INTO student(std_id,std_name,std_sex,std_birth,std_in,std_address) VALUES (29,'钱二','男','1993-01-29','2011-09-01','江苏省南京市雨花台区');
INSERT INTO student(std_id,std_name,std_sex,std_birth,std_in,std_address) VALUES (30,'钱三','男','1993-01-30','2011-09-01','江苏省南京市雨花台区');
INSERT INTO student(std_id,std_name,std_sex,std_birth,std_in,std_address) VALUES (31,'钱四','男','1993-02-01','2011-09-01','江苏省南京市雨花台区');
INSERT INTO student(std_id,std_name,std_sex,std_birth,std_in,std_address) VALUES (32,'钱五','男','1993-02-02','2011-09-01','江苏省南京市雨花台区');
INSERT INTO student(std_id,std_name,std_sex,std_birth,std_in,std_address) VALUES (33,'钱六','男','1993-02-03','2011-09-01','江苏省南京市雨花台区');
INSERT INTO student(std_id,std_name,std_sex,std_birth,std_in,std_address) VALUES (34,'钱七','男','1993-02-04','2011-09-01','江苏省南京市雨花台区');
INSERT INTO student(std_id,std_name,std_sex,std_birth,std_in,std_address) VALUES (35,'钱八','男','1993-02-05','2011-09-01','江苏省南京市雨花台区');
INSERT INTO student(std_id,std_name,std_sex,std_birth,std_in,std_address) VALUES (36,'钱九','男','1993-02-06','2011-09-01','江苏省南京市雨花台区');
INSERT INTO student(std_id,std_name,std_sex,std_birth,std_in,std_address) VALUES (37,'吴一','男','1993-02-07','2011-09-01','江苏省南京市雨花台区');
INSERT INTO student(std_id,std_name,std_sex,std_birth,std_in,std_address) VALUES (38,'吴二','男','1993-02-08','2011-09-01','江苏省南京市雨花台区');
INSERT INTO student(std_id,std_name,std_sex,std_birth,std_in,std_address) VALUES (39,'吴三','男','1993-02-09','2011-09-01','江苏省南京市雨花台区');
```

```
INSERT INTO student(std_id,std_name,std_sex,std_birth,std_in,std_address) VALUES (40,'吴四','男','1993-02-10','2011-09-01','江苏省南京市雨花台区');
INSERT INTO student(std_id,std_name,std_sex,std_birth,std_in,std_address) VALUES (41,'吴五','男','1993-02-11','2011-09-01','江苏省南京市雨花台区');
INSERT INTO student(std_id,std_name,std_sex,std_birth,std_in,std_address) VALUES (42,'吴六','男','1993-02-12','2011-09-01','江苏省南京市雨花台区');
INSERT INTO student(std_id,std_name,std_sex,std_birth,std_in,std_address) VALUES (43,'吴七','男','1993-02-13','2011-09-01','江苏省南京市雨花台区');
INSERT INTO student(std_id,std_name,std_sex,std_birth,std_in,std_address) VALUES (44,'吴八','男','1993-02-14','2011-09-01','江苏省南京市雨花台区');
INSERT INTO student(std_id,std_name,std_sex,std_birth,std_in,std_address) VALUES (45,'吴九','男','1993-02-15','2011-09-01','江苏省南京市雨花台区');
INSERT INTO student(std_id,std_name,std_sex,std_birth,std_in,std_address) VALUES (46,'柳一','男','1993-02-16','2011-09-01','江苏省南京市雨花台区');
INSERT INTO student(std_id,std_name,std_sex,std_birth,std_in,std_address) VALUES (47,'柳二','男','1993-02-17','2011-09-01','江苏省南京市雨花台区');
INSERT INTO student(std_id,std_name,std_sex,std_birth,std_in,std_address) VALUES (48,'柳三','男','1993-02-18','2011-09-01','江苏省南京市雨花台区');
INSERT INTO student(std_id,std_name,std_sex,std_birth,std_in,std_address) VALUES (49,'柳四','男','1993-02-19','2011-09-01','江苏省南京市雨花台区');
INSERT INTO student(std_id,std_name,std_sex,std_birth,std_in,std_address) VALUES (50,'柳五','男','1993-02-20','2011-09-01','江苏省南京市雨花台区');
```

截图如下：



步骤 2 向 teacher 表中插入数据。

```
INSERT INTO teacher(tec_id,tec_name,tec_job,tec_sex,tec_age,tec_in) VALUES (1,'张一','讲师','男',35,'2009-07-01');
INSERT INTO teacher(tec_id,tec_name,tec_job,tec_sex,tec_age,tec_in) VALUES (2,'张二','讲师','男',35,'2009-07-01');
INSERT INTO teacher(tec_id,tec_name,tec_job,tec_sex,tec_age,tec_in) VALUES (3,'张三','讲师','男',35,'2009-07-01');
INSERT INTO teacher(tec_id,tec_name,tec_job,tec_sex,tec_age,tec_in) VALUES (4,'张四','讲师','男',35,'2009-07-01');
INSERT INTO teacher(tec_id,tec_name,tec_job,tec_sex,tec_age,tec_in) VALUES (5,'张五','讲师','男',35,'2009-07-01');
INSERT INTO teacher(tec_id,tec_name,tec_job,tec_sex,tec_age,tec_in) VALUES (6,'张六','讲师','男',35,'2009-07-01');
INSERT INTO teacher(tec_id,tec_name,tec_job,tec_sex,tec_age,tec_in) VALUES (7,'张七','讲师','男',35,'2009-07-01');
```

```
INSERT INTO teacher(tec_id,tec_name,tec_job,tec_sex,tec_age,tec_in) VALUES (8,'张八','讲师','男',35,'2009-07-01');
INSERT INTO teacher(tec_id,tec_name,tec_job,tec_sex,tec_age,tec_in) VALUES (9,'张九','讲师','男',35,'2009-07-01');
INSERT INTO teacher(tec_id,tec_name,tec_job,tec_sex,tec_age,tec_in) VALUES (10,'李一','讲师','男',35,'2009-07-01');
INSERT INTO teacher(tec_id,tec_name,tec_job,tec_sex,tec_age,tec_in) VALUES (11,'李二','讲师','男',35,'2009-07-01');
INSERT INTO teacher(tec_id,tec_name,tec_job,tec_sex,tec_age,tec_in) VALUES (12,'李三','讲师','男',35,'2009-07-01');
INSERT INTO teacher(tec_id,tec_name,tec_job,tec_sex,tec_age,tec_in) VALUES (13,'李四','讲师','男',35,'2009-07-01');
INSERT INTO teacher(tec_id,tec_name,tec_job,tec_sex,tec_age,tec_in) VALUES (14,'李五','讲师','男',35,'2009-07-01');
INSERT INTO teacher(tec_id,tec_name,tec_job,tec_sex,tec_age,tec_in) VALUES (15,'李六','讲师','男',35,'2009-07-01');
INSERT INTO teacher(tec_id,tec_name,tec_job,tec_sex,tec_age,tec_in) VALUES (16,'李七','讲师','男',35,'2009-07-01');
INSERT INTO teacher(tec_id,tec_name,tec_job,tec_sex,tec_age,tec_in) VALUES (17,'李八','讲师','男',35,'2009-07-01');
INSERT INTO teacher(tec_id,tec_name,tec_job,tec_sex,tec_age,tec_in) VALUES (18,'李九','讲师','男',35,'2009-07-01');
INSERT INTO teacher(tec_id,tec_name,tec_job,tec_sex,tec_age,tec_in) VALUES (19,'王一','讲师','男',35,'2009-07-01');
INSERT INTO teacher(tec_id,tec_name,tec_job,tec_sex,tec_age,tec_in) VALUES (20,'王二','讲师','男',35,'2009-07-01');
INSERT INTO teacher(tec_id,tec_name,tec_job,tec_sex,tec_age,tec_in) VALUES (21,'王三','讲师','男',35,'2009-07-01');
INSERT INTO teacher(tec_id,tec_name,tec_job,tec_sex,tec_age,tec_in) VALUES (22,'王四','讲师','男',35,'2009-07-01');
INSERT INTO teacher(tec_id,tec_name,tec_job,tec_sex,tec_age,tec_in) VALUES (23,'王五','讲师','男',35,'2009-07-01');
INSERT INTO teacher(tec_id,tec_name,tec_job,tec_sex,tec_age,tec_in) VALUES (24,'王六','讲师','男',35,'2009-07-01');
INSERT INTO teacher(tec_id,tec_name,tec_job,tec_sex,tec_age,tec_in) VALUES (25,'王七','讲师','男',35,'2009-07-01');
INSERT INTO teacher(tec_id,tec_name,tec_job,tec_sex,tec_age,tec_in) VALUES (26,'王八','讲师','男',35,'2009-07-01');
INSERT INTO teacher(tec_id,tec_name,tec_job,tec_sex,tec_age,tec_in) VALUES (27,'王九','讲师','男',35,'2009-07-01');
INSERT INTO teacher(tec_id,tec_name,tec_job,tec_sex,tec_age,tec_in) VALUES (28,'钱一','讲师','男',35,'2009-07-01');
INSERT INTO teacher(tec_id,tec_name,tec_job,tec_sex,tec_age,tec_in) VALUES (29,'钱二','讲师','男',35,'2009-07-01');
INSERT INTO teacher(tec_id,tec_name,tec_job,tec_sex,tec_age,tec_in) VALUES (30,'钱三','讲师','男',35,'2009-07-01');
INSERT INTO teacher(tec_id,tec_name,tec_job,tec_sex,tec_age,tec_in) VALUES (31,'钱四','讲师','男',35,'2009-07-01');
INSERT INTO teacher(tec_id,tec_name,tec_job,tec_sex,tec_age,tec_in) VALUES (32,'钱五','讲师','男',35,'2009-07-01');
INSERT INTO teacher(tec_id,tec_name,tec_job,tec_sex,tec_age,tec_in) VALUES (33,'钱六','讲师','男',35,'2009-07-01');
```

```
INSERT INTO teacher(tec_id,tec_name,tec_job,tec_sex,tec_age,tec_in) VALUES (34,'钱七','讲师','男',35,'2009-07-01');
INSERT INTO teacher(tec_id,tec_name,tec_job,tec_sex,tec_age,tec_in) VALUES (35,'钱八','讲师','男',35,'2009-07-01');
INSERT INTO teacher(tec_id,tec_name,tec_job,tec_sex,tec_age,tec_in) VALUES (36,'钱九','讲师','男',35,'2009-07-01');
INSERT INTO teacher(tec_id,tec_name,tec_job,tec_sex,tec_age,tec_in) VALUES (37,'吴一','讲师','男',35,'2009-07-01');
INSERT INTO teacher(tec_id,tec_name,tec_job,tec_sex,tec_age,tec_in) VALUES (38,'吴二','讲师','男',35,'2009-07-01');
INSERT INTO teacher(tec_id,tec_name,tec_job,tec_sex,tec_age,tec_in) VALUES (39,'吴三','讲师','男',35,'2009-07-01');
INSERT INTO teacher(tec_id,tec_name,tec_job,tec_sex,tec_age,tec_in) VALUES (40,'吴四','讲师','男',35,'2009-07-01');
INSERT INTO teacher(tec_id,tec_name,tec_job,tec_sex,tec_age,tec_in) VALUES (41,'吴五','讲师','男',35,'2009-07-01');
INSERT INTO teacher(tec_id,tec_name,tec_job,tec_sex,tec_age,tec_in) VALUES (42,'吴六','讲师','男',35,'2009-07-01');
INSERT INTO teacher(tec_id,tec_name,tec_job,tec_sex,tec_age,tec_in) VALUES (43,'吴七','讲师','男',35,'2009-07-01');
INSERT INTO teacher(tec_id,tec_name,tec_job,tec_sex,tec_age,tec_in) VALUES (44,'吴八','讲师','男',35,'2009-07-01');
INSERT INTO teacher(tec_id,tec_name,tec_job,tec_sex,tec_age,tec_in) VALUES (45,'吴九','讲师','男',35,'2009-07-01');
INSERT INTO teacher(tec_id,tec_name,tec_job,tec_sex,tec_age,tec_in) VALUES (46,'柳一','讲师','男',35,'2009-07-01');
INSERT INTO teacher(tec_id,tec_name,tec_job,tec_sex,tec_age,tec_in) VALUES (47,'柳二','讲师','男',35,'2009-07-01');
INSERT INTO teacher(tec_id,tec_name,tec_job,tec_sex,tec_age,tec_in) VALUES (48,'柳三','讲师','男',35,'2009-07-01');
INSERT INTO teacher(tec_id,tec_name,tec_job,tec_sex,tec_age,tec_in) VALUES (49,'柳四','讲师','男',35,'2009-07-01');
INSERT INTO teacher(tec_id,tec_name,tec_job,tec_sex,tec_age,tec_in) VALUES (50,'柳五','讲师','男',35,'2009-07-01');
```

截图如下：



步骤 3 向 class 表插入数据。

```
INSERT INTO class(cla_id,cla_name,cla_teacher) VALUES (1,'计算机',1);
```

```
INSERT INTO class(cla_id,cla_name,cla_teacher) VALUES (2,'自动化',3);
INSERT INTO class(cla_id,cla_name,cla_teacher) VALUES (3,'飞行器设计',5);
INSERT INTO class(cla_id,cla_name,cla_teacher) VALUES (4,'大学物理',7);
INSERT INTO class(cla_id,cla_name,cla_teacher) VALUES (5,'高等数学',9);
INSERT INTO class(cla_id,cla_name,cla_teacher) VALUES (6,'大学化学',12);
INSERT INTO class(cla_id,cla_name,cla_teacher) VALUES (7,'表演',14);
INSERT INTO class(cla_id,cla_name,cla_teacher) VALUES (8,'服装设计',16);
INSERT INTO class(cla_id,cla_name,cla_teacher) VALUES (9,'工业设计',18);
INSERT INTO class(cla_id,cla_name,cla_teacher) VALUES (10,'金融学',21);
INSERT INTO class(cla_id,cla_name,cla_teacher) VALUES (11,'医学',23);
INSERT INTO class(cla_id,cla_name,cla_teacher) VALUES (12,'土木工程',25);
INSERT INTO class(cla_id,cla_name,cla_teacher) VALUES (13,'机械',27);
INSERT INTO class(cla_id,cla_name,cla_teacher) VALUES (14,'建筑学',29);
INSERT INTO class(cla_id,cla_name,cla_teacher) VALUES (15,'经济学',32);
INSERT INTO class(cla_id,cla_name,cla_teacher) VALUES (16,'财务管理',34);
INSERT INTO class(cla_id,cla_name,cla_teacher) VALUES (17,'人力资源',36);
INSERT INTO class(cla_id,cla_name,cla_teacher) VALUES (18,'力学',38);
INSERT INTO class(cla_id,cla_name,cla_teacher) VALUES (19,'人工智能',41);
INSERT INTO class(cla_id,cla_name,cla_teacher) VALUES (20,'会计',45);
```

截图如下：



步骤 4 向 school_department 表插入数据。

```
INSERT INTO school_department(depart_id,depart_name,depart_teacher) VALUES (1,'计算机学院',2);
INSERT INTO school_department(depart_id,depart_name,depart_teacher) VALUES (2,'自动化学院',4);
INSERT INTO school_department(depart_id,depart_name,depart_teacher) VALUES (3,'航空宇航学院',6);
INSERT INTO school_department(depart_id,depart_name,depart_teacher) VALUES (4,'艺术学院',8);
INSERT INTO school_department(depart_id,depart_name,depart_teacher) VALUES (5,'理学院',11);
INSERT INTO school_department(depart_id,depart_name,depart_teacher) VALUES (6,'人工智能学院',13);
INSERT INTO school_department(depart_id,depart_name,depart_teacher) VALUES (7,'工学院',15);
INSERT INTO school_department(depart_id,depart_name,depart_teacher) VALUES (8,'管理学院',17);
INSERT INTO school_department(depart_id,depart_name,depart_teacher) VALUES (9,'农学院',22);
INSERT INTO school_department(depart_id,depart_name,depart_teacher) VALUES (10,'医学院',28);
```

截图如下：



步骤 5 向 course 表插入数据。

```
INSERT INTO course(cor_id,cor_name,cor_type,credit) VALUES (1,'数据库系统概论','必修',3);
INSERT INTO course(cor_id,cor_name,cor_type,credit) VALUES (2,'艺术设计概论','选修',1);
INSERT INTO course(cor_id,cor_name,cor_type,credit) VALUES (3,'力学制图','必修',4);
INSERT INTO course(cor_id,cor_name,cor_type,credit) VALUES (4,'飞行器设计历史','选修',1);
INSERT INTO course(cor_id,cor_name,cor_type,credit) VALUES (5,'马克思主义','必修',2);
INSERT INTO course(cor_id,cor_name,cor_type,credit) VALUES (6,'大学历史','必修',2);
INSERT INTO course(cor_id,cor_name,cor_type,credit) VALUES (7,'人力资源管理理论','必修',2.5);
INSERT INTO course(cor_id,cor_name,cor_type,credit) VALUES (8,'线性代数','必修',4);
INSERT INTO course(cor_id,cor_name,cor_type,credit) VALUES (9,'JAVA 程序设计','必修',3);
INSERT INTO course(cor_id,cor_name,cor_type,credit) VALUES (10,'操作系统','必修',4);
INSERT INTO course(cor_id,cor_name,cor_type,credit) VALUES (11,'计算机组成原理','必修',3);
INSERT INTO course(cor_id,cor_name,cor_type,credit) VALUES (12,'自动化设计理论','必修',2);
INSERT INTO course(cor_id,cor_name,cor_type,credit) VALUES (13,'情绪表演','必修',2.5);
INSERT INTO course(cor_id,cor_name,cor_type,credit) VALUES (14,'茶学历史','选修',1);
INSERT INTO course(cor_id,cor_name,cor_type,credit) VALUES (15,'艺术论','必修',1.5);
INSERT INTO course(cor_id,cor_name,cor_type,credit) VALUES (16,'机器学习','必修',3);
INSERT INTO course(cor_id,cor_name,cor_type,credit) VALUES (17,'数据挖掘','选修',2);
INSERT INTO course(cor_id,cor_name,cor_type,credit) VALUES (18,'图像识别','必修',3);
INSERT INTO course(cor_id,cor_name,cor_type,credit) VALUES (19,'解剖学','必修',4);
INSERT INTO course(cor_id,cor_name,cor_type,credit) VALUES (20,'3D max','选修',2);
```

截图如下：



1.6.3 数据查询

1.6.3.1 单表查询

查询 B 校课程信息表的所有信息。

```
SELECT * from course;
```

截图如下：



SQL执行记录 消息 结果集1 X

以下是SELECT * from course;的执行结果集 该表不可编辑。

	cor_id	cor_name	cor_type	credit
1	1	数据库系统概论	必修	3
2	2	艺术设计概论	选修	1
3	3	力学制图	必修	4
4	4	飞行器设计历史	选修	1
5	5	马克思主义	必修	2

1.6.3.2 条件查询

在教师信息表中查询教师编号大于 45 的老师的入学年份。

```
SELECT tec_id, tec_in FROM teacher WHERE tec_id>45;
```

截图如下：



SQL执行记录 消息 结果集1 X

以下是SELECT tec_id, tec_in FROM teacher WHERE tec_id>45;的执行结果集 该表不可编辑。

	tec_id	tec_in
1	46	2009-07-01
2	47	2009-07-01
3	48	2009-07-01
4	49	2009-07-01
5	50	2009-07-01

查询 B 校中所有选修的课程的信息。

```
SELECT * FROM course WHERE cor_type='选修';
```

截图如下：

执行SQL(F8)

格式化(F9)

执行计划(F6)

我的SQL

1

SELECT * FROM course WHERE cor_type='选修';

SQL执行记录

消息

结果集1

以下是SELECT * FROM course WHERE cor_type='选修'的执行结果集

该表不可编辑。

	cor_id	cor_name	cor_type	credit
1	2	艺术设计概论	选修	1
2	4	飞行器设计历史	选修	1
3	14	茶学历史	选修	1
4	17	数据挖掘	选修	2
5	20	3D max	选修	2

1.6.3.3 修改数据

修改/更新课程信息表数据。

```

UPDATE course SET cor_name='C 语言程序设计',cor_type='必修',credit=3.5 WHERE cor_id=1;
SELECT * FROM course WHERE cor_id=1;

```

截图如下：

执行SQL(F8)

格式化(F9)

执行计划(F6)

我的SQL

1 UPDATE course SET cor_name='C语言程序设计',cor_type='必修',credit=3.5 WHERE cor_id=1;

2 SELECT * FROM course WHERE cor_id=1;

3

SQL执行记录

消息

结果集1 X

以下是SELECT * FROM course WHERE cor_id=1的执行结果集

该表不可编辑。

	cor_id	cor_name	cor_type	credit
1	1	C语言程序设计	必修	3.5

1.6.3.4 删除指定数据

在 B 校中删除教师编号 8 和 15 所管理的院系。

```

DELETE FROM school_department WHERE depart_teacher=8 OR depart_teacher=15;
SELECT * FROM school_department;

```

截图如下：

执行SQL(F8)

格式化(F9)

执行计划(F6)

我的SQL

1

2

3

4

DELETE FROM school_department WHERE depart_teacher=8 OR depart_teacher=15;

SELECT * FROM school_department;

SQL执行记录

消息

结果集1 X

以下是SELECT * FROM school_department的执行结果集

该表不可编辑。

	depart_id	depart_name	depart_teacher
1	1	计算机学院	2
2	2	自动化学院	4
3	3	航空宇航学院	6
4	5	理学院	11
5	6	人工智能学院	13

本实验结束。